TECHNICAL PROJECT DOCUMENTATION · CYBERSECURITY LAB

AI-Assisted Security Alert Triage

Wazuh → n8n → AI Agent → Slack

`Webhook` → `Extract Alert Context` → `IF` → `AI Agent` → `Slack`

STACK

Wazuh n8n OpenRouter AI Agent Slack Windows Kali Linux

Public edition — credentials, IP/MAC addresses and webhook URLs redacted

CONTENTS

Table of Contents

SECTION 01	**n8n Workflow**	**3**
Step 1	Create a new workflow	3
Step 2	Add the Webhook trigger	3
Step 4	Confirm the test event in the Webhook	5
Step 5	Extract Alert Context	6
Step 6	Filter alerts with an IF node	8
Step 7	AI Agent and OpenRouter	9
Step 8	Add the prompt	11
Step 9	Create the Slack app	13
Step 10	Send the alert to Slack	18
Step 11	Custom n8n integration on Wazuh	23
Step 12	Restart the Wazuh manager	26
Step 13	PowerShell test script and Slack channel	27
Step 14	Publish the workflow	28
Step 15	Run the test	29
SECTION 02	**Problems**	**31**
Problem 1	PowerShell execution policy	31
Problem 2	Alert level thresholds	32
Problem 3	Process creation and command-line auditing	36
SECTION 03	**Important Note**	**40**
	What is a webhook?	40
	Why POST?	41
	Overall	41
	Explain URLS	42

SECTION 01

n8n Workflow

STEP 1 Create a new workflow

- Click to the new workflow

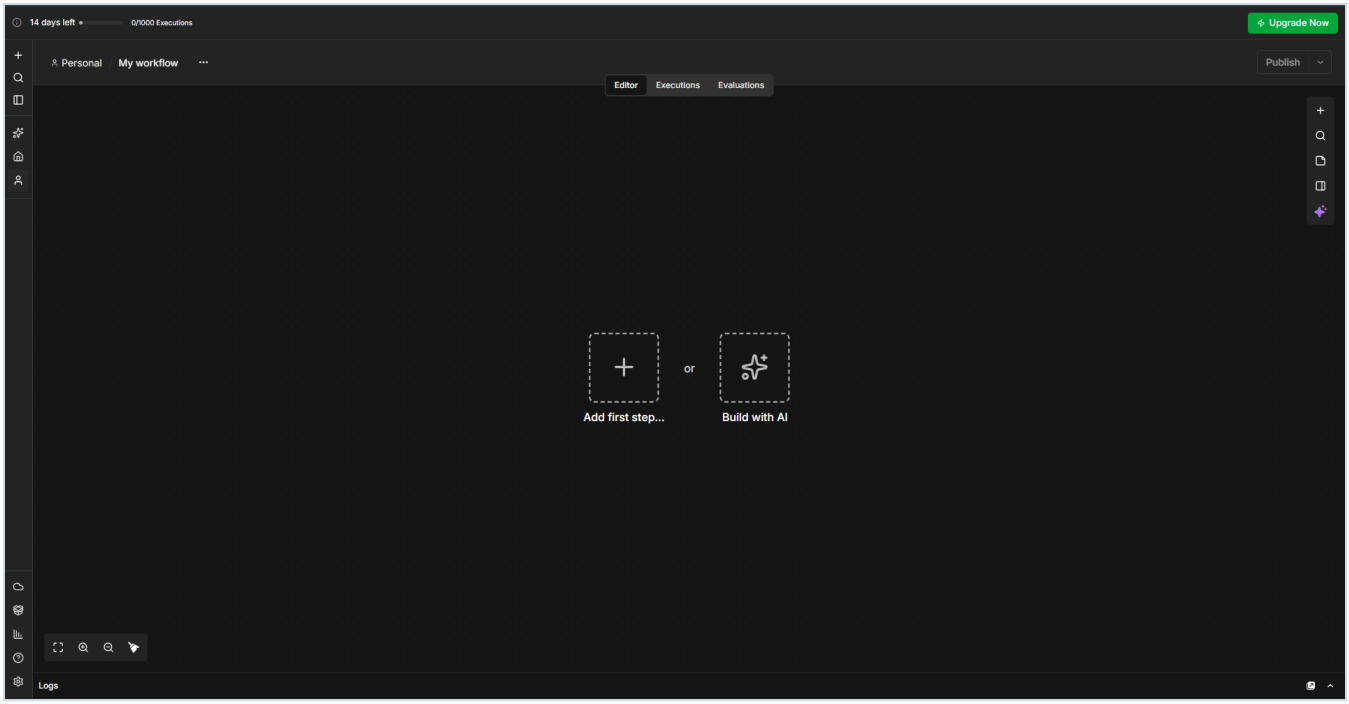


FIGURE 1

STEP 2 Add the Webhook trigger

- First i will click add first step and choose the webhook it is an endpoint who receive HTTP request & inside Method Choose : POST because it is appropriate method when Wazuh is sending alert data to n8n.

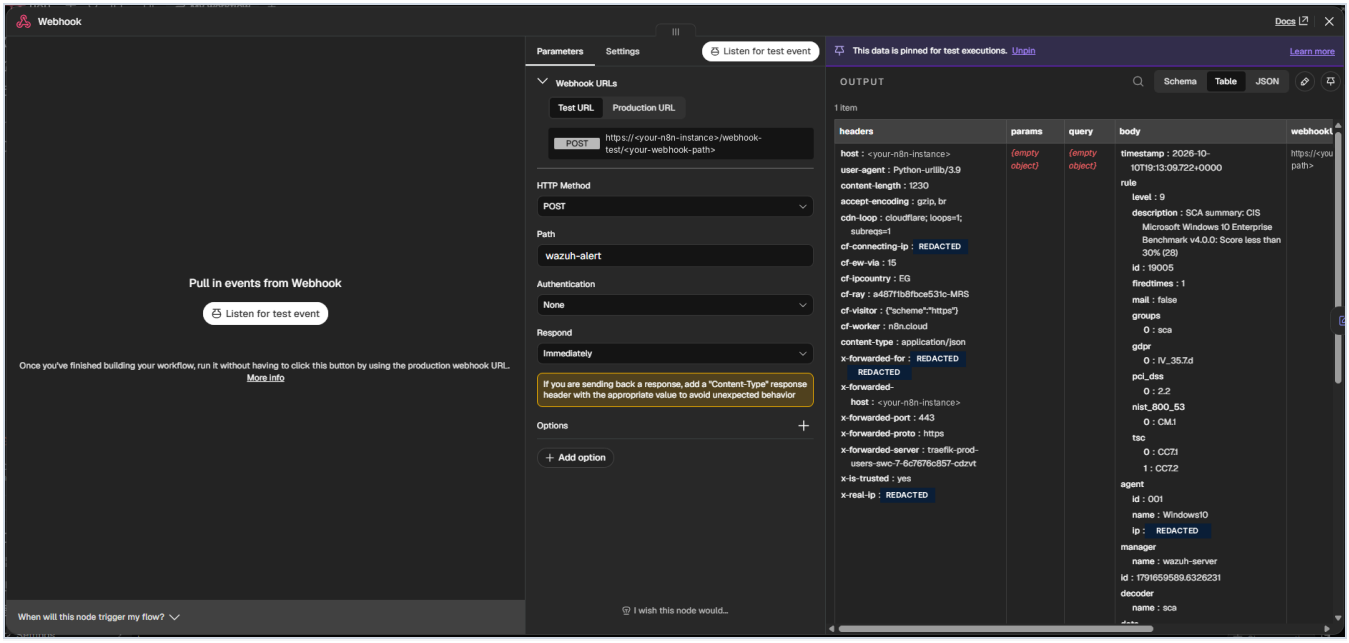


FIGURE 2

- this step i will check that the webhook is receive any HTTP request, i open CMD an copy the test URL from webhook (https://<your-n8n-instance>/webhook-test/<your-webhook-path>) and click for list for event inside webhook

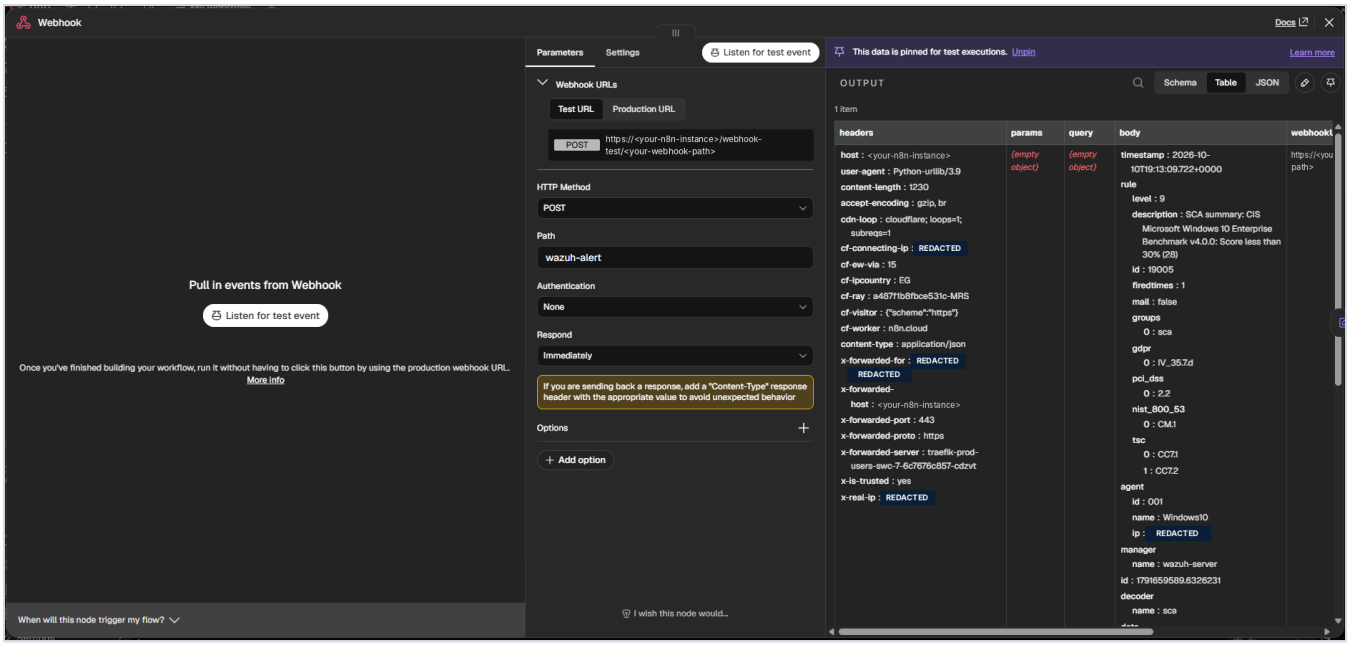


FIGURE 3

- paste it inside in CMD as command :

```
CMD
C:\Users\Ibrahim>curl -X POST "https://<your-n8n-instance>/webhook-test/<your-webhook
-path>" -H "Content-Type: application/json" -d "{"rule":{"id":"100002","level":10,"descri
ption":"Suspicious PowerShell activity"},"agent":{"name":"WIN11-LAB"}}"
```

- In CMD will appear : `{"message":"Workflow was started"}`

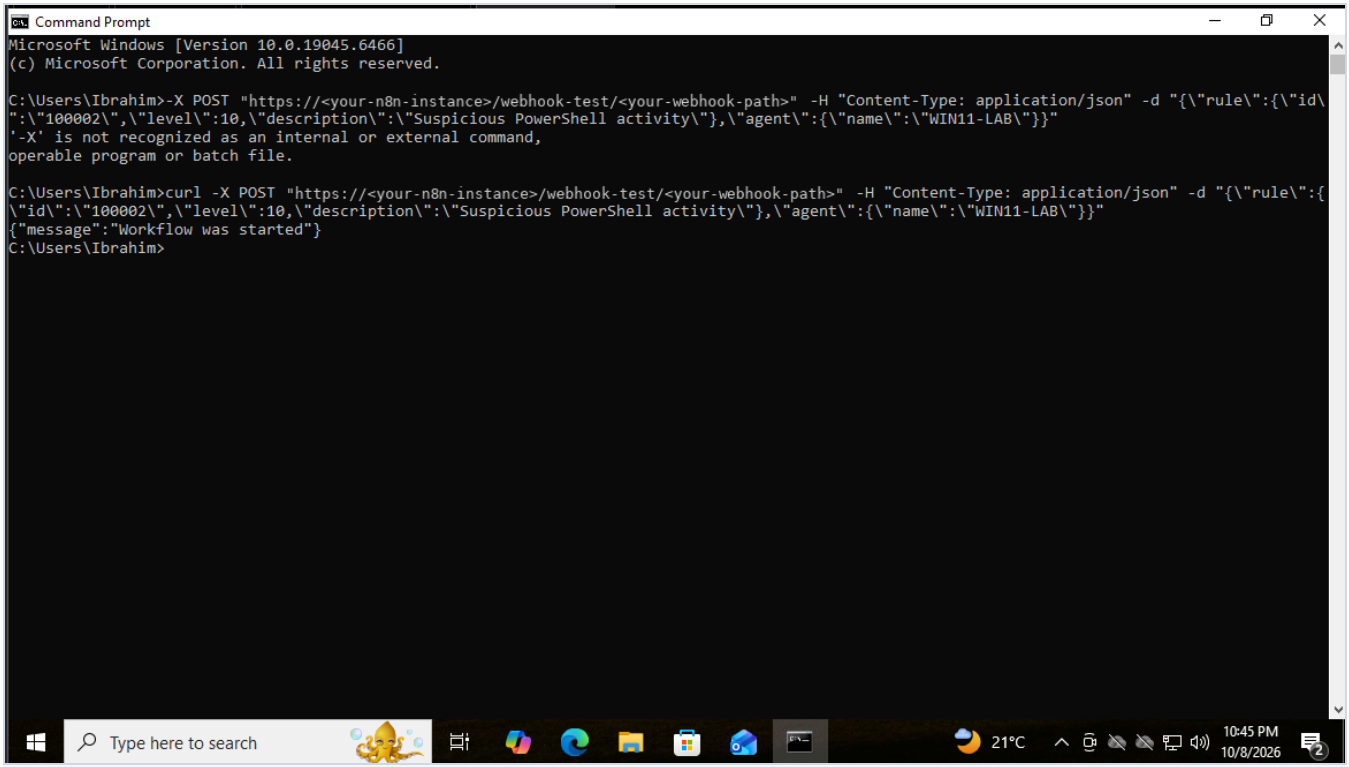


FIGURE 4

STEP 4 Confirm the test event in the Webhook

- In Webhook will appear

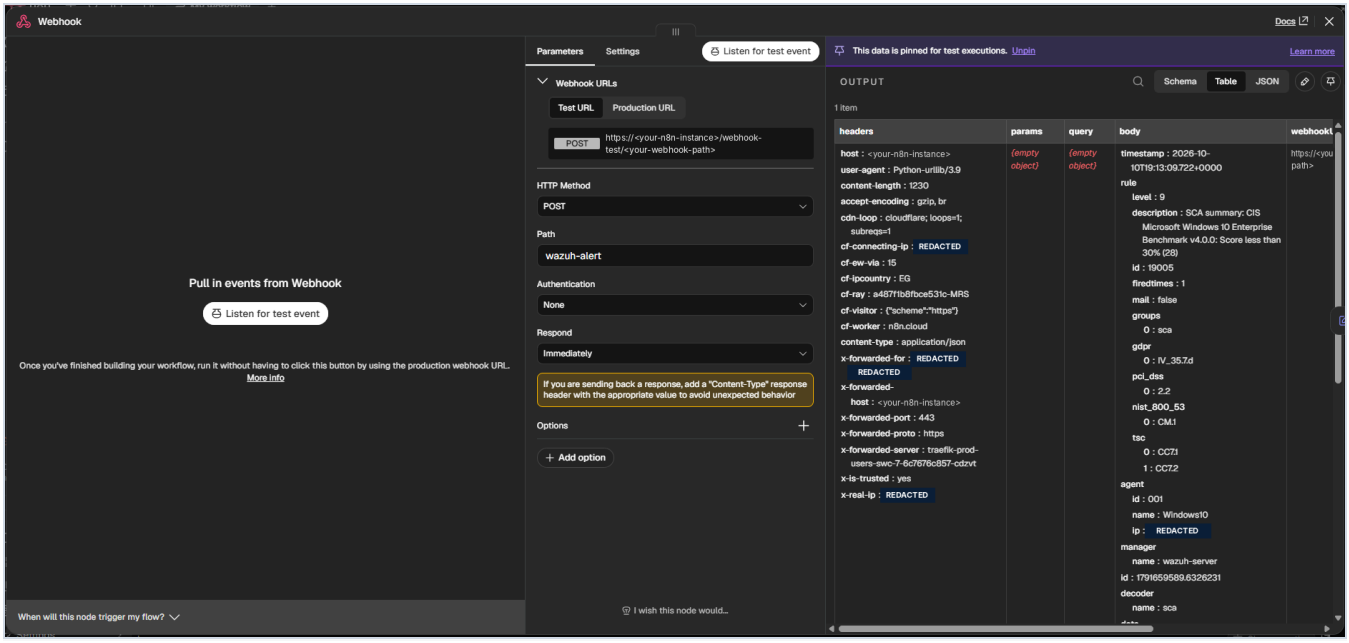


FIGURE 5

STEP 5 Extract Alert Context

- Wazuh will send a JSON file containing a large amount of information, but I don't need all of this information to be sent to the AI. Instead, I will first extract the important information from the JSON file. Then, I will add another node called **"Extract Alert Context."**
- To create this node, I will first search for **Edit Fields (Set)** in n8n and rename it to **Extract Alert Context**. Then, I will click on this node, select **Add Field**, and add the following fields:
  - `rule_id`
  - `rule_level`
  - `description`
  - `event_id`
  - `Provider`
  - `User`
  - `Process`
  - `Command Line`
  - `Target File`
  - `Process ID`
  - `Parent Process ID`
  - `Raw Message`
- If Wazuh sends a large alert containing a lot of information, I will extract only the **important information** that is needed for the AI.
  - The **Extract Alert Context** node will select the key fields from the alert as same above
  - This way, instead of sending the **entire large Wazuh alert** to the AI, I will send only the **relevant alert context**. This reduces unnecessary information and allows the AI to focus on the important details of the security alert.

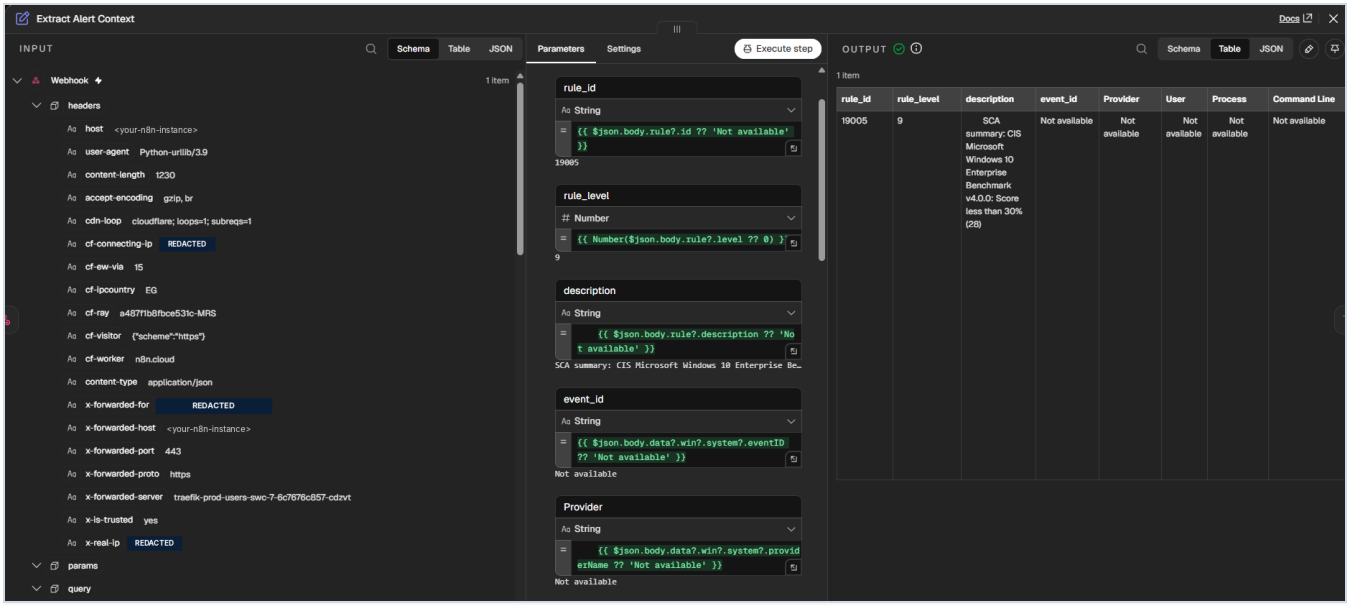


FIGURE 6

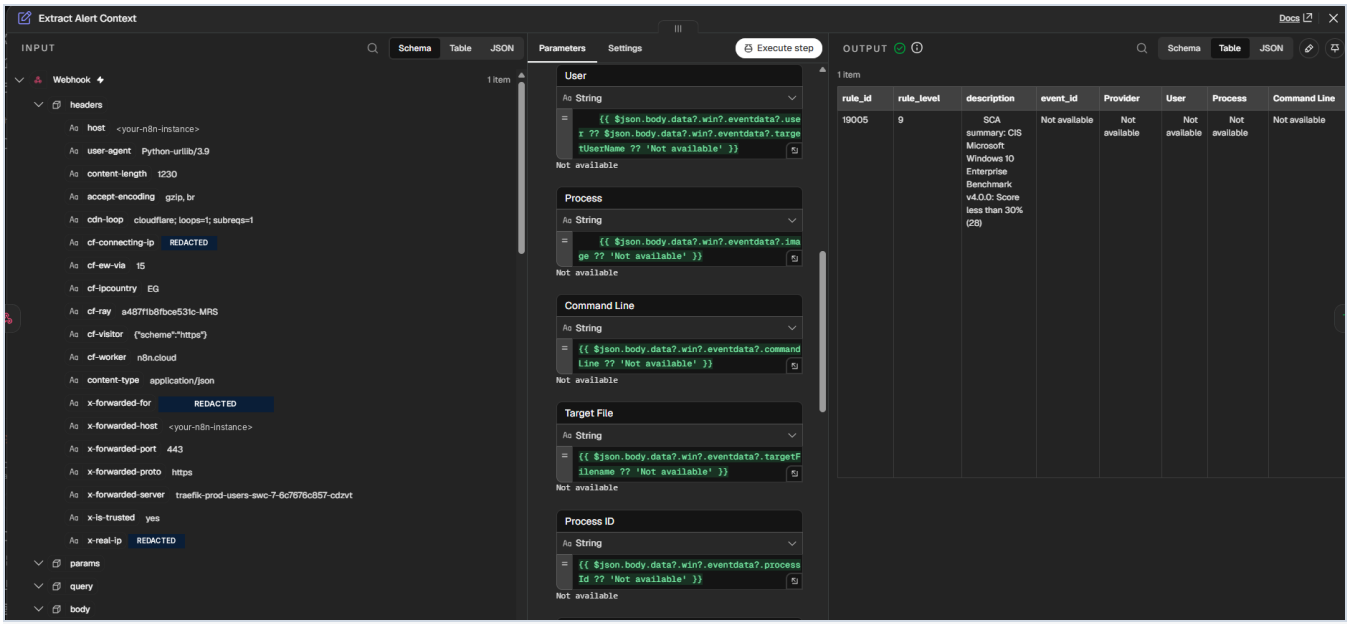


FIGURE 7

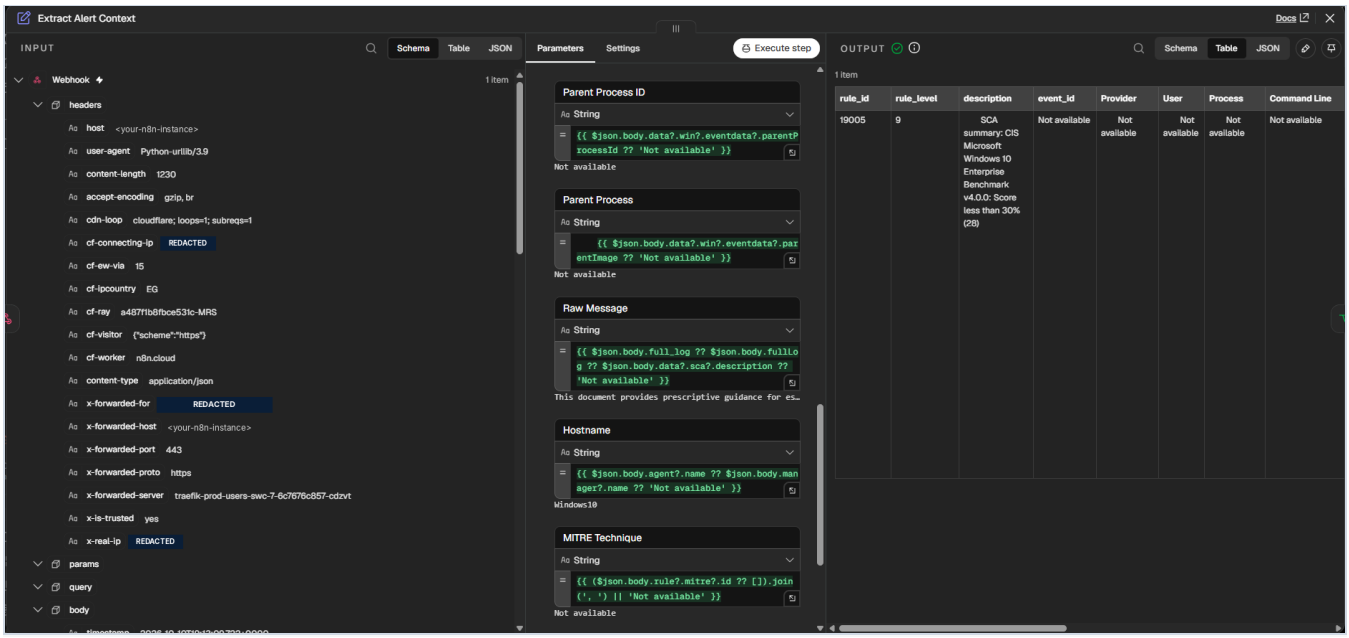


FIGURE 8

STEP 6 Filter alerts with an IF node

- Before sending anything to the LLM, I must add a **filter** because it doesn't make sense to send every Wazuh alert to an expensive or limited LLM. Not every event requires AI analysis.
- For this, I will add another node called **IF Node**.
- Inside the **Parameters**, I will configure the condition using the `rule_level` field that I extracted in the **Extract Alert Context** node.
- The condition will be : `rule_level` → greater than or equal to → `9`
- The condition is `rule_level >= 9`. This means alerts with a **Wazuh rule level of 9 or higher** are forwarded to the AI Agent for analysis. Alerts below level 9 follow the False Branch. An alert with `rule_level = 9` successfully passed the condition, as demonstrated in the IF Node test screenshot (kept in the project repository as `if-node-threshold-test.png`; not reproduced in this PDF).
- Since Wazuh rule levels range from **1 to 15** (not 1 to 9), using `>= 9` means we are selecting the higher-severity alerts for AI analysis.
- Figure 10 on the next page was captured while the workflow was being built and still shows the original condition `{{ $('Webhook').item.json.body.rule.level }}` greater than or equal to `10`. The threshold was later lowered to 9 (see **Problem 2 – Alert level thresholds**).

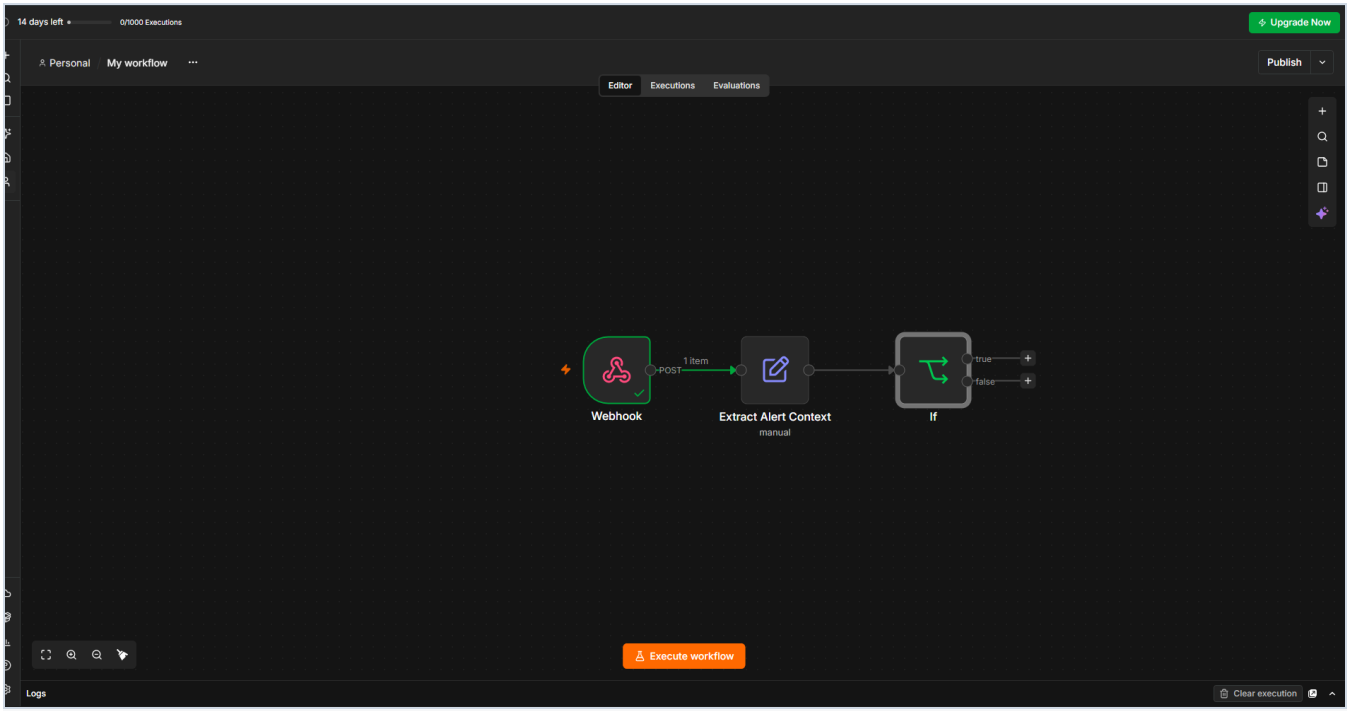


FIGURE 9

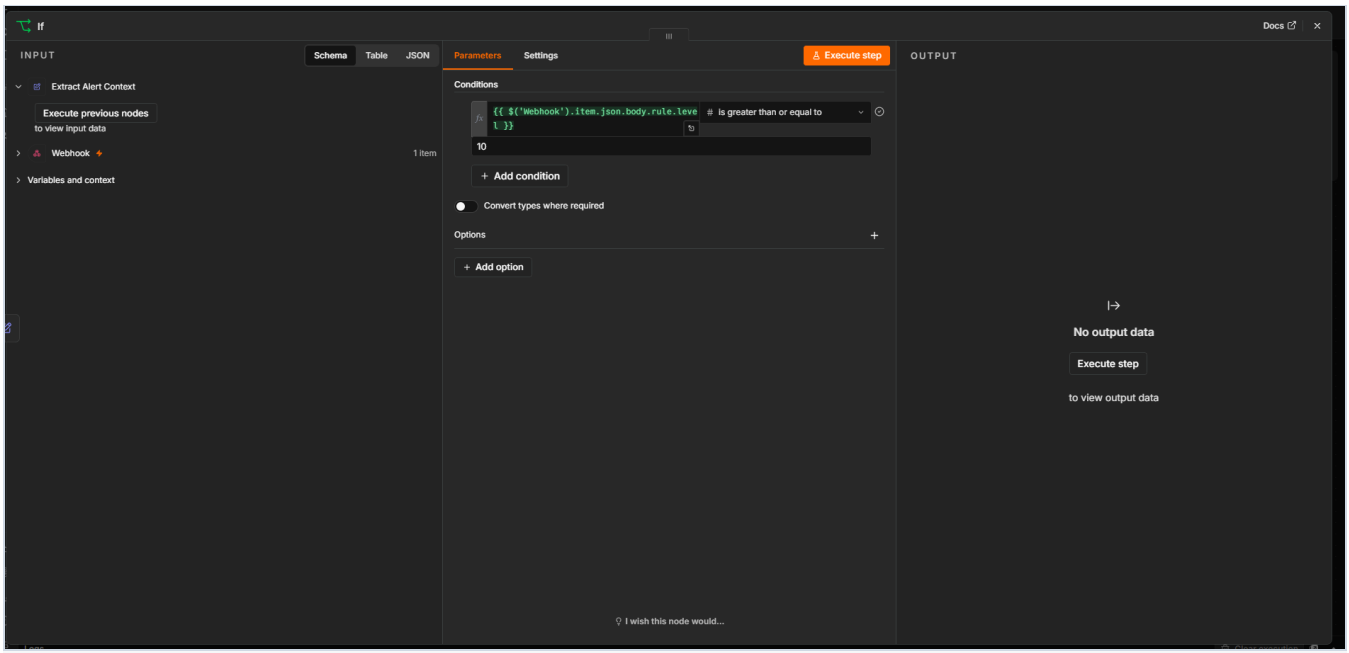


FIGURE 10

STEP 7 AI Agent and OpenRouter

- Then I will add an **AI Agent** node and connect the **Yes** output from the previous **IF Node** to the **AI Agent**.

- The AI Agent needs a model to analyze the alert data. Here, I will use **OpenRouter** because it provides a single API that allows me to access many different AI models from different providers. This gives me the flexibility to choose the most suitable model for analyzing the Wazuh alerts without having to integrate each provider separately.

For OpenRouter

- i add a new key and named it as n8n-ai-agent-lab
- Key : REDACTED

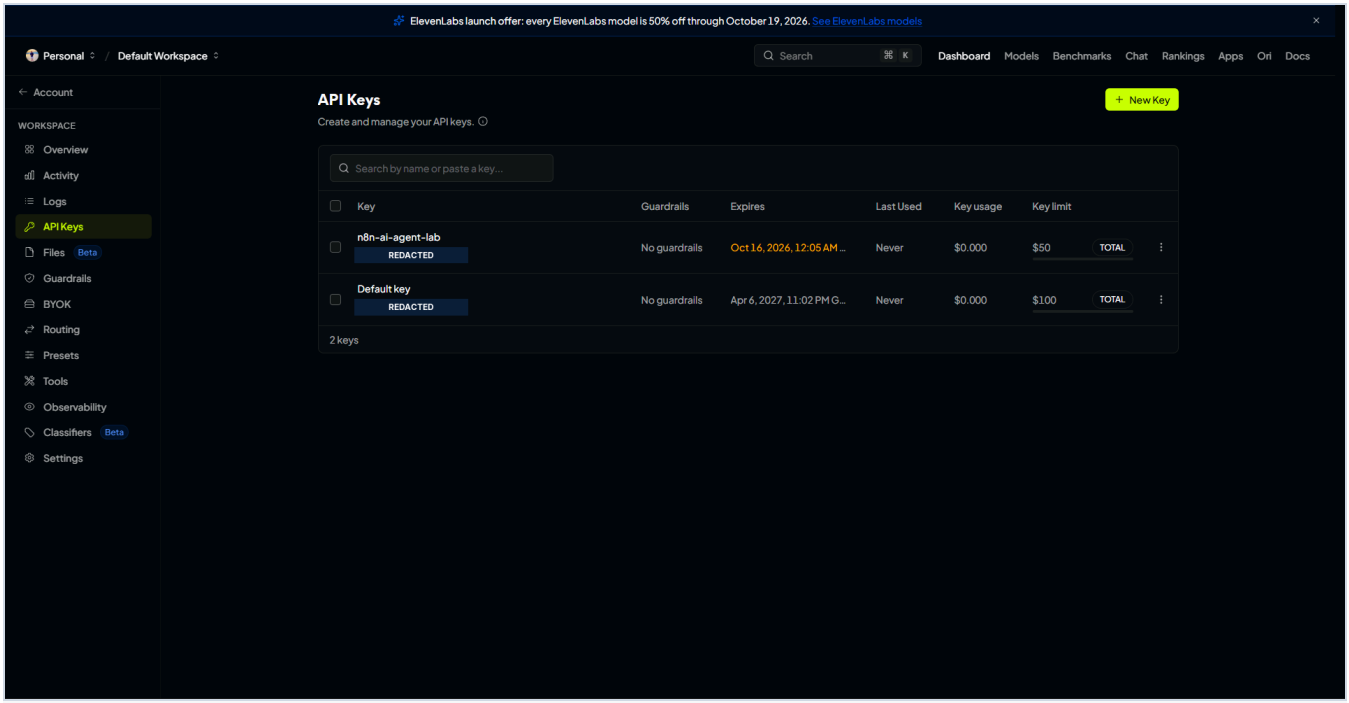


FIGURE 11

- After creating an API key on the **OpenRouter** platform, I will return to **n8n**. Then, I will click on the **Chat Model** inside the **AI Agent** node and search for **OpenRouter**.
- Next, I will select **Connect OpenRouter** and enter the API key that I created on the OpenRouter platform. This will connect n8n to OpenRouter and allow the AI Agent to use the available models through the OpenRouter API & i will use a model called (`deepseek/deepseek-v4-flash`)

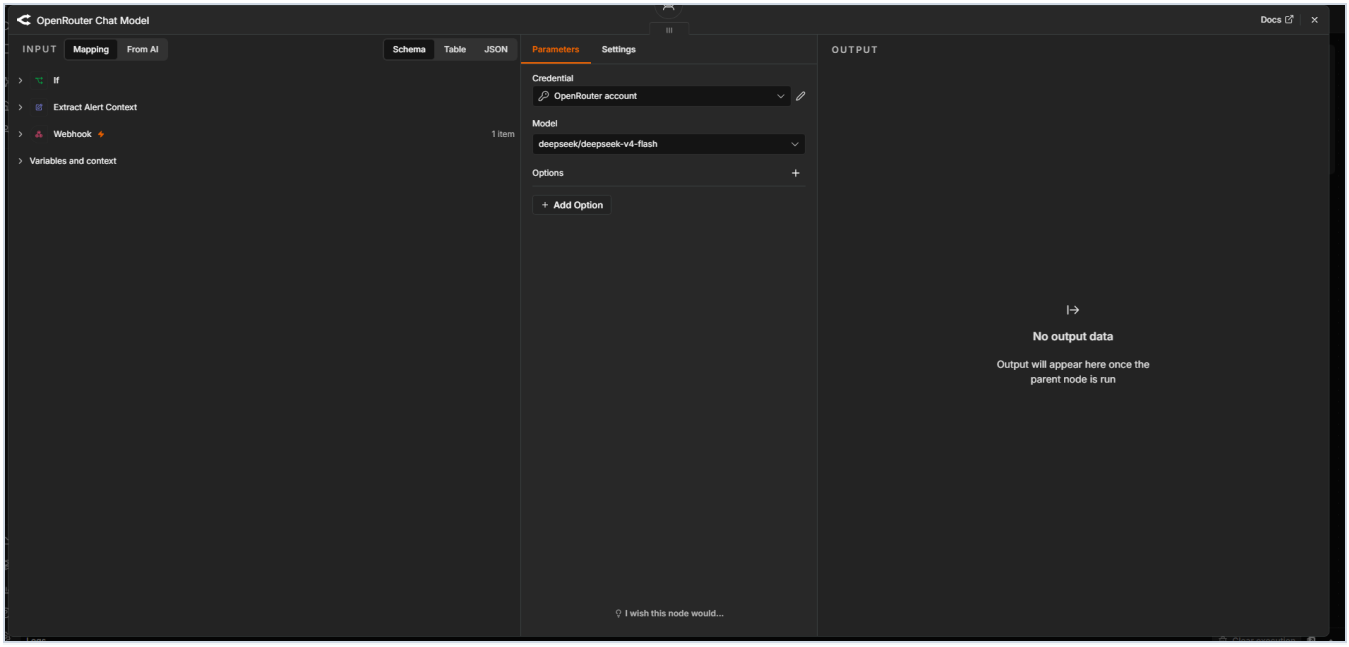


FIGURE 12

STEP 8 Add the prompt

- Then, I will click on the **AI Agent**, click on the **Source for Prompt**, and select **Define Flow**. Then, I will add my prompt to the **Prompt** field.

Prompt

AI AGENT · PROMPT

```
You are a SOC analyst.

Analyze this security alert:

Rule ID: {{$json.rule_id}} Rule Level: {{$json.rule_level}}
Description: {{$json.description}} Hostname: {{$json["Hostname"]}}
Event ID: {{$json.event_id}} Provider: {{$json.Provider}}
User: {{$json["User"]}} Process: {{$json["Process"]}}
Command Line: {{$json["Command Line"]}} Parent Process: {{$json["Parent Process"]}}
Target File: {{$json["Target File"]}} Process ID: {{$json["Process ID"]}}
Parent Process ID: {{$json["Parent Process ID"]}}
MITRE Technique: {{$json["MITRE Technique"]}}
Raw Message: {{$json["Raw Message"]}}

Create a concise SOC alert for Slack.

Rules:
• Only use facts explicitly present in the provided alert fields.
```

- Do not invent MITRE ATT&CK techniques, Event IDs, processes, users, files, commands, or attacker behavior.
- Do not assume a specific Windows or Sysmon event type.
- Different events contain different fields; missing fields are normal.
- If a field is missing, do not infer its value.
- Clearly separate observed evidence from analysis.
- If the event only proves file creation, describe file creation.
- If it proves process execution, describe process execution.
- If intent cannot be determined from the evidence, say so.
- Do not recommend containment unless the evidence clearly justifies it.
- Do not assign or calculate severity. Severity is handled separately by Wazuh/n8n.
- Always include the observed command line if available.
- Do not paraphrase the observed command line.
- If the command contains encoded, obfuscated, or transformed content and you can reliably decode or interpret it, include the decoded/interpreted result.
- If you cannot reliably decode or interpret it, write "Not available".
- Keep the response concise.
- Use Slack-compatible mrkdwn.
- Do not use double asterisks.

Use exactly this structure:

🚨 *Security Alert*

Host: {{$json["Hostname"]}} *User:* {{$json["User"]}}
Process: {{$json["Process"]}} *Severity Level:* {{$json.rule_level}}

What happened
<1-2 concise sentences based only on observed evidence>

Observed command
<exact command line as provided, or "Not available">

Decoded / interpreted content
<decoded or interpreted content if reliably available, otherwise "Not available">

Why it matters
<1 concise analytical sentence>

MITRE ATT&CK
<technique ID and name, or "Not specified">

Recommended action
<1 concise investigation step>

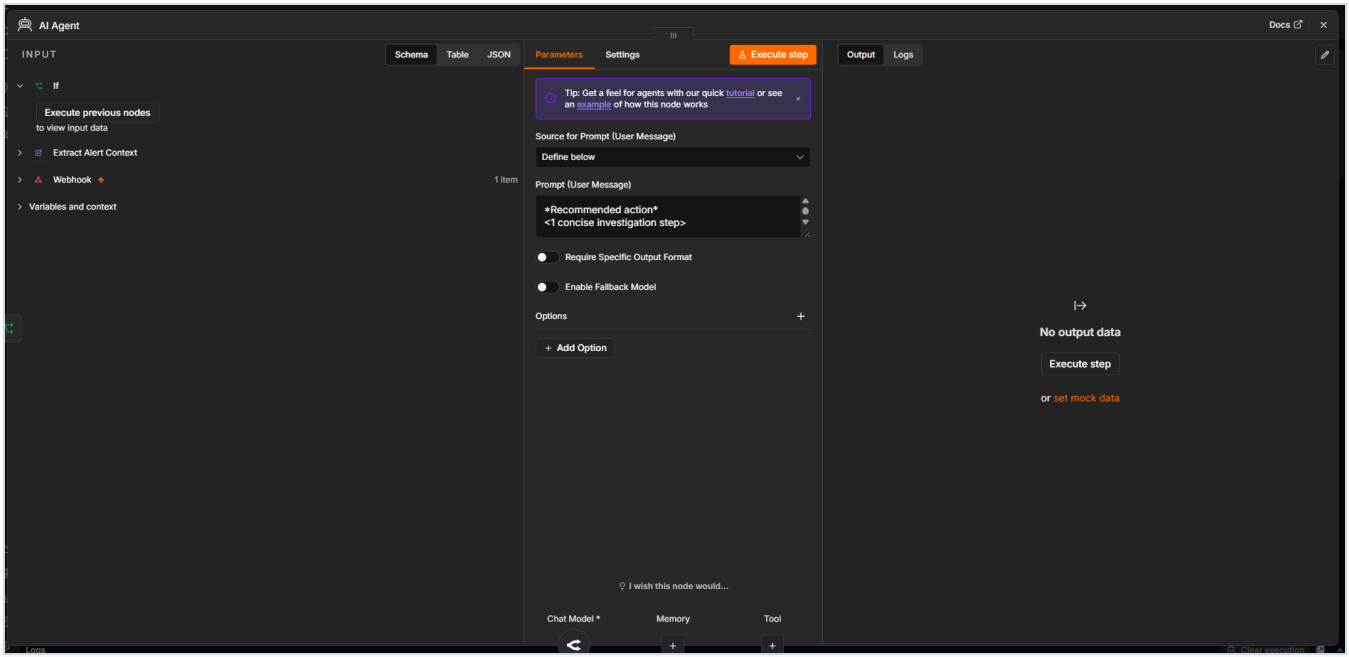


FIGURE 13

STEP 9 Create the Slack app

- To make this prompt work, I must connect the **AI Agent** to **Slack**.
- Then, I will click **Create App**, select **Blank App**, and name the app (**n8n-security-lab**) & Choose the workplace called (**n8n-security-lab**)

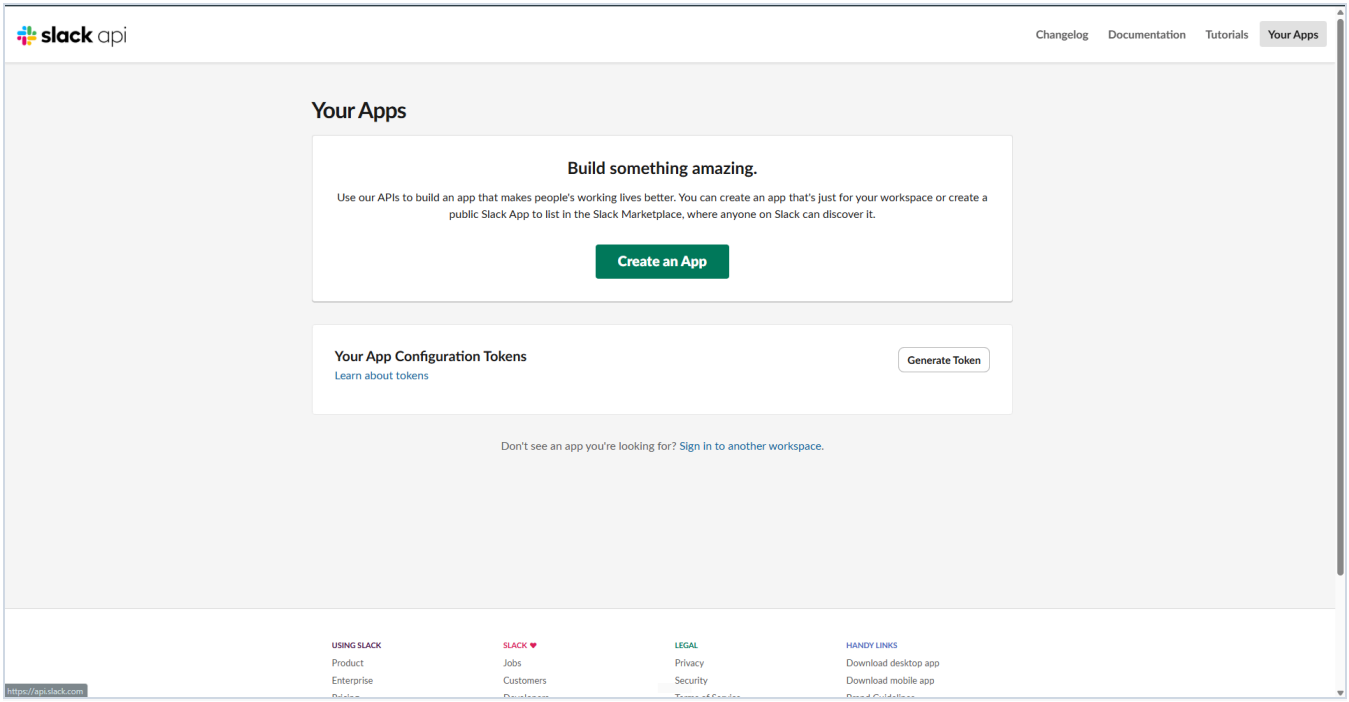


FIGURE 14

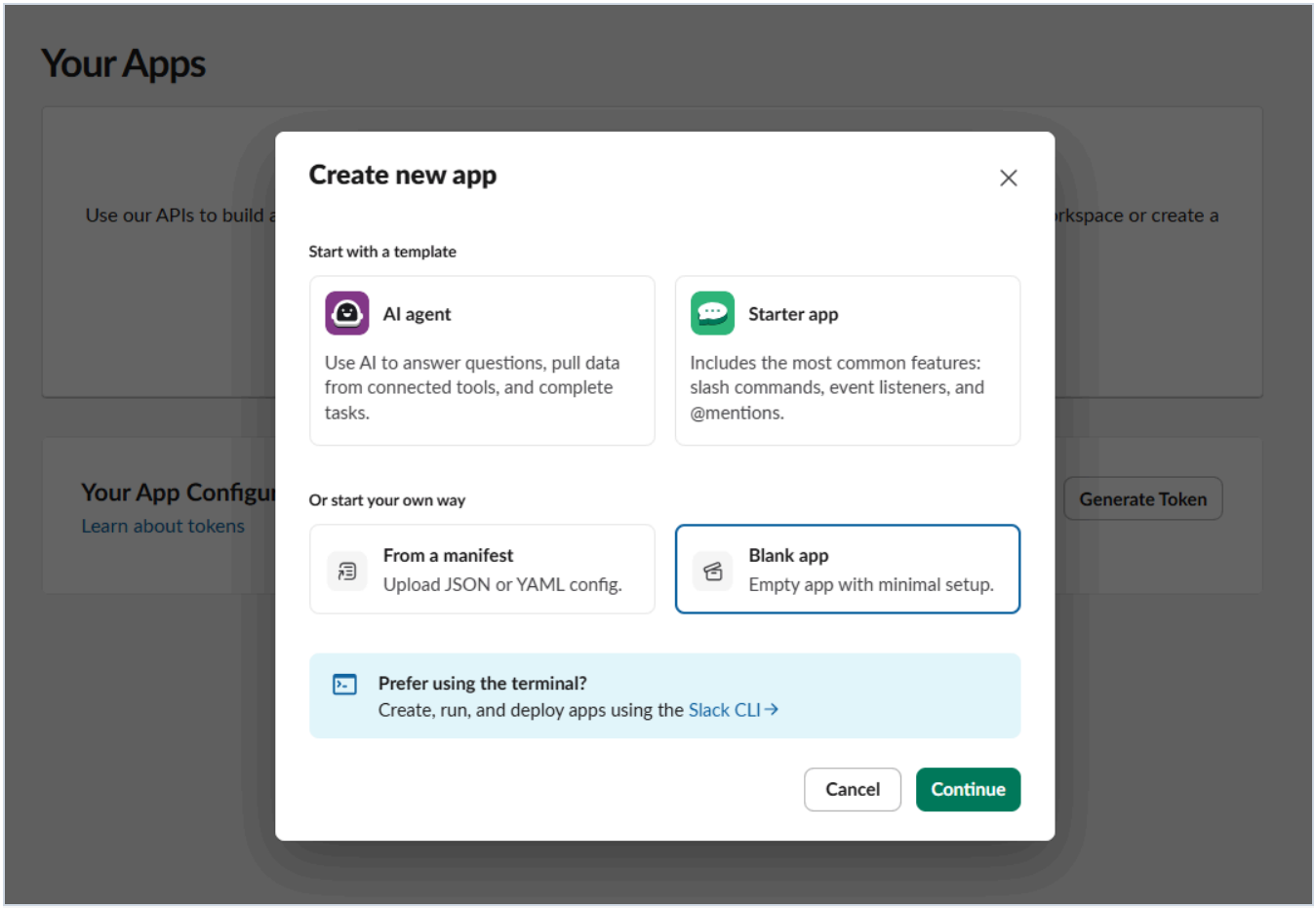


FIGURE 15

- Then, I will search for **Slack** on Google to create a new workspace and name it (**n8n-security-lab**).

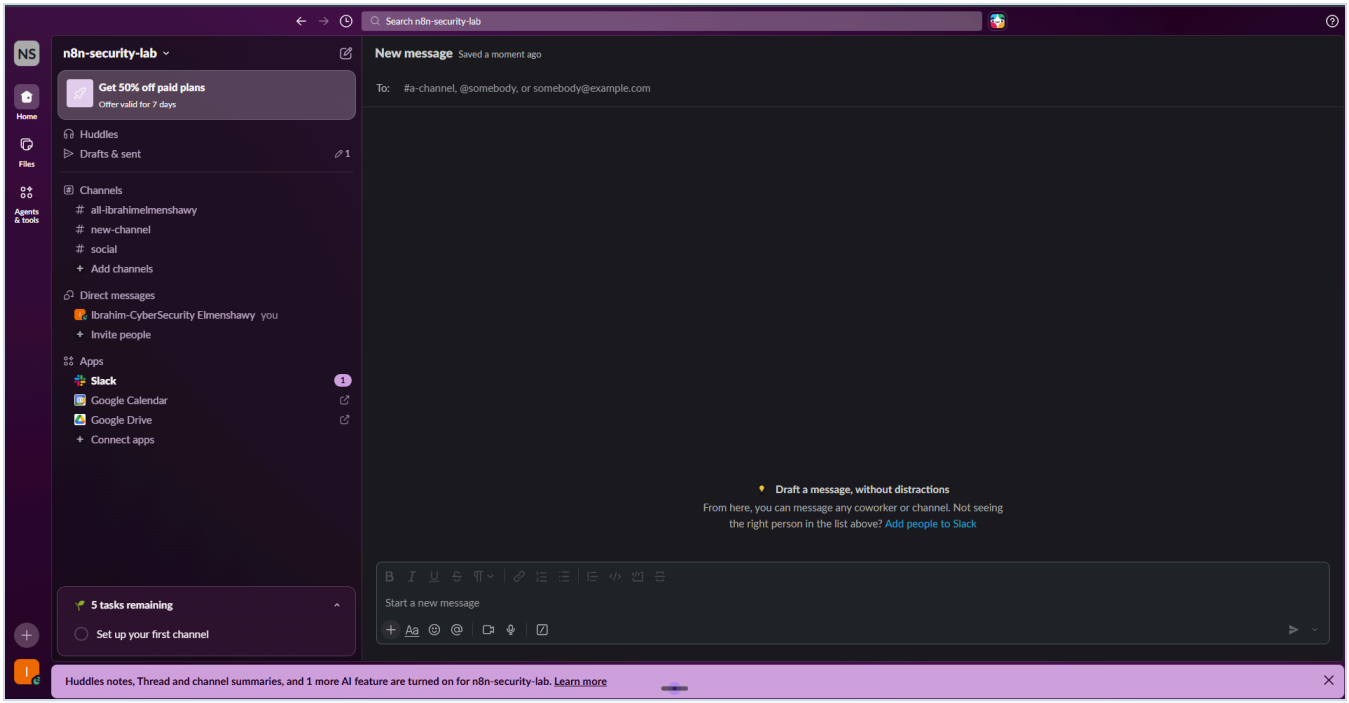


FIGURE 16

- After I create the app, the app page will appear. Then, I will focus on (**Add an OAuth Scope**) and search for (`chat:write`). This permission allows the bot I created to send messages to Slack.

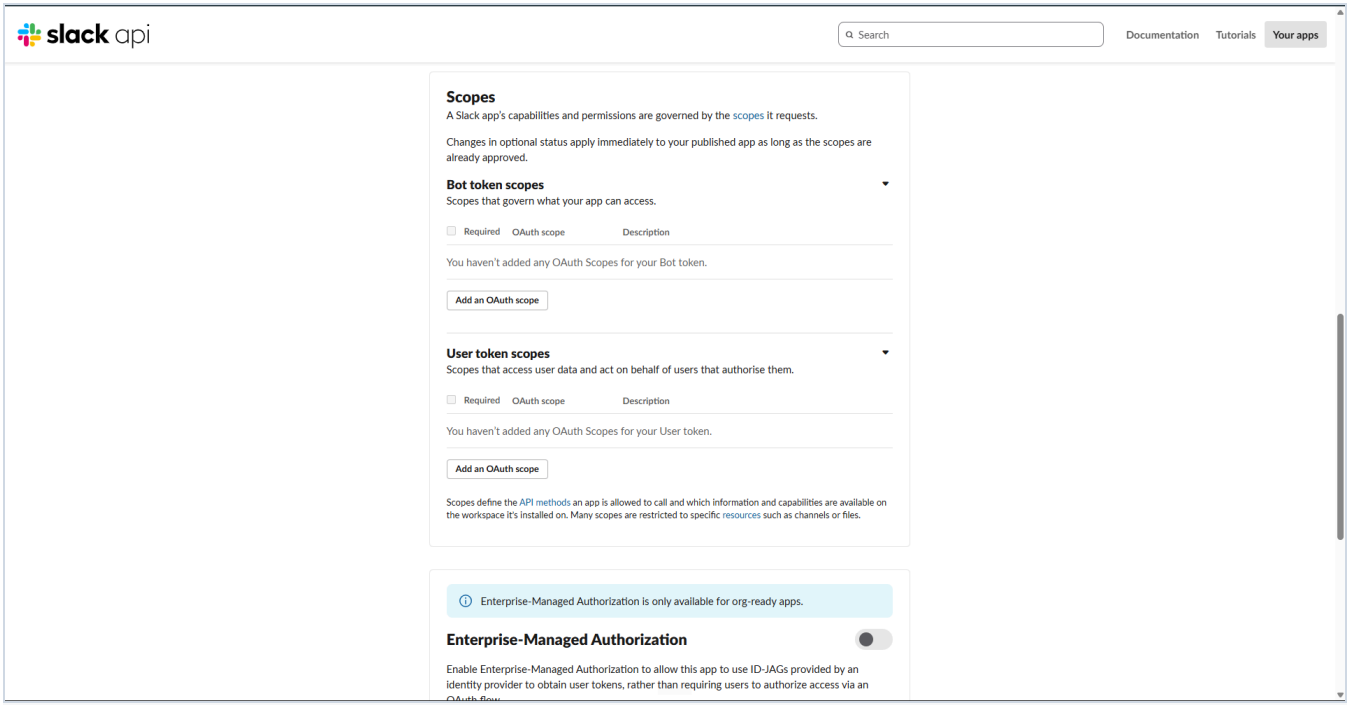


FIGURE 17

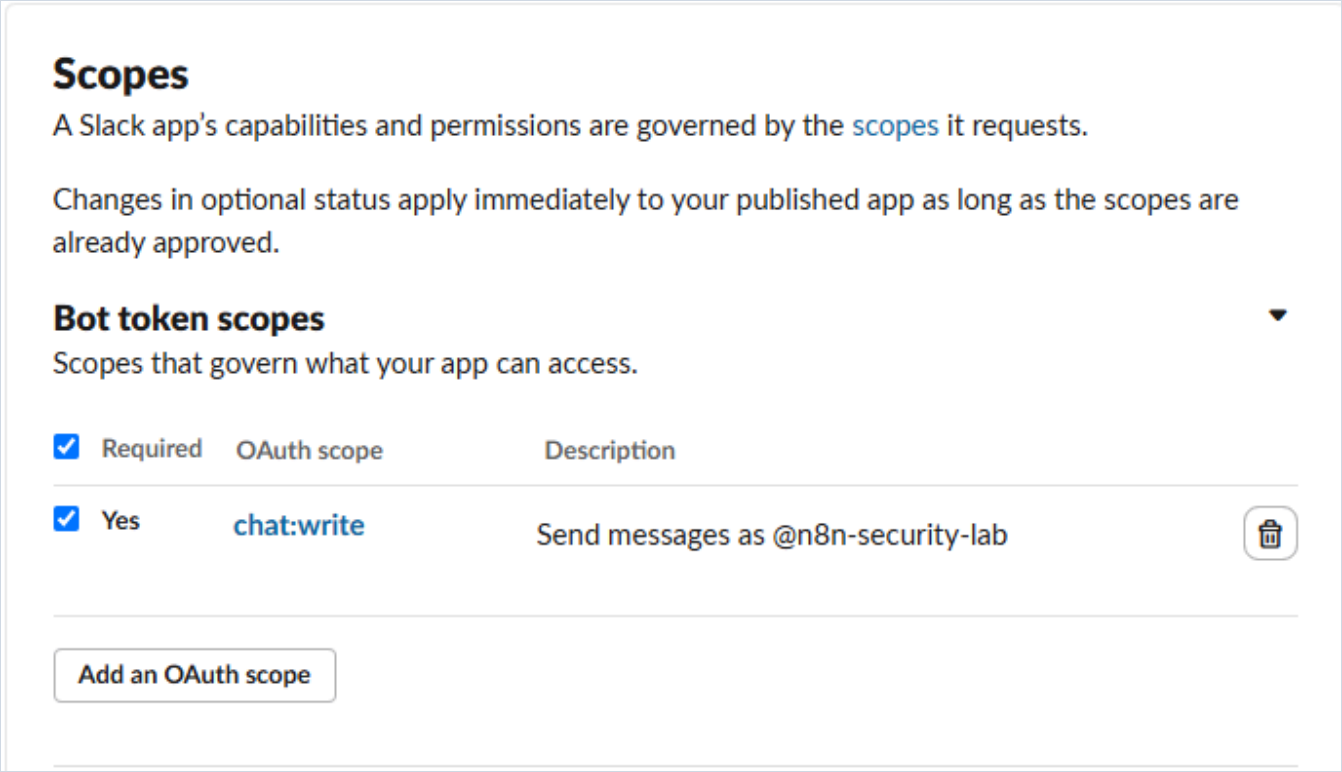


FIGURE 18

- Then, I will click **Install App**, and then click **Install to (n8n-security-lab)** & then Click allow and it give us tokens
- Bot User OAuth Token : REDACTED

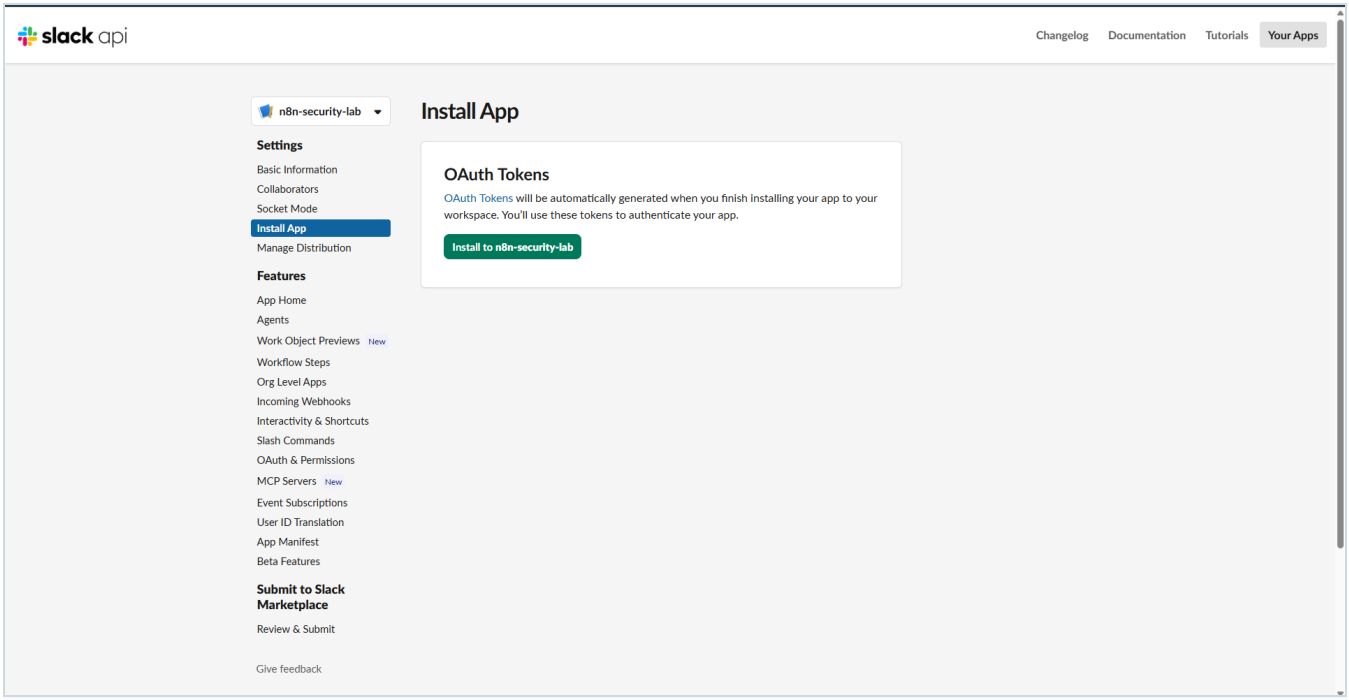


FIGURE 19

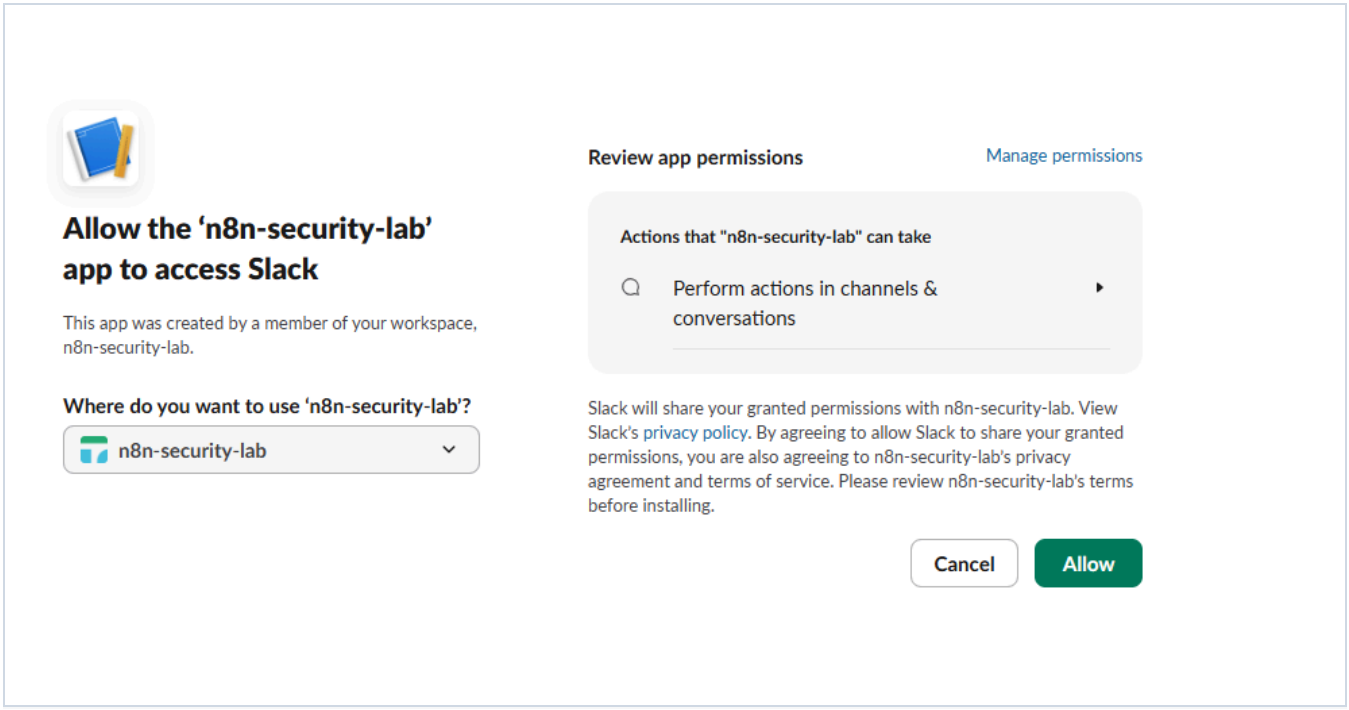


FIGURE 20

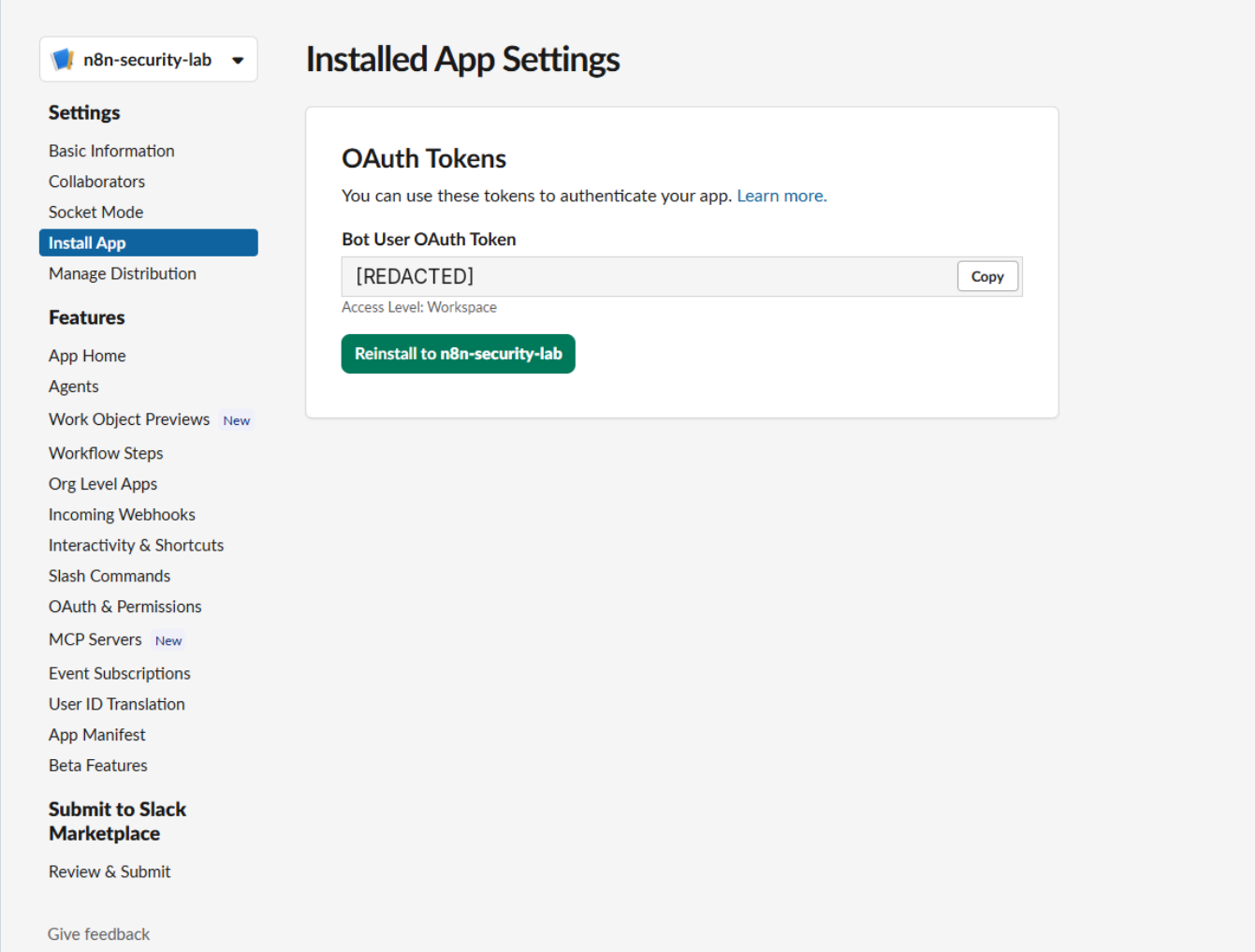


FIGURE 21

STEP 10 Send the alert to Slack

- I will copy the token that I get from **Slack** and return to **n8n**. Then, I will add another node called **Slack** and search for an operation called **Send Message**. I will click **Connect to Slack**, paste the token, and then enter the **Signing Secret**.

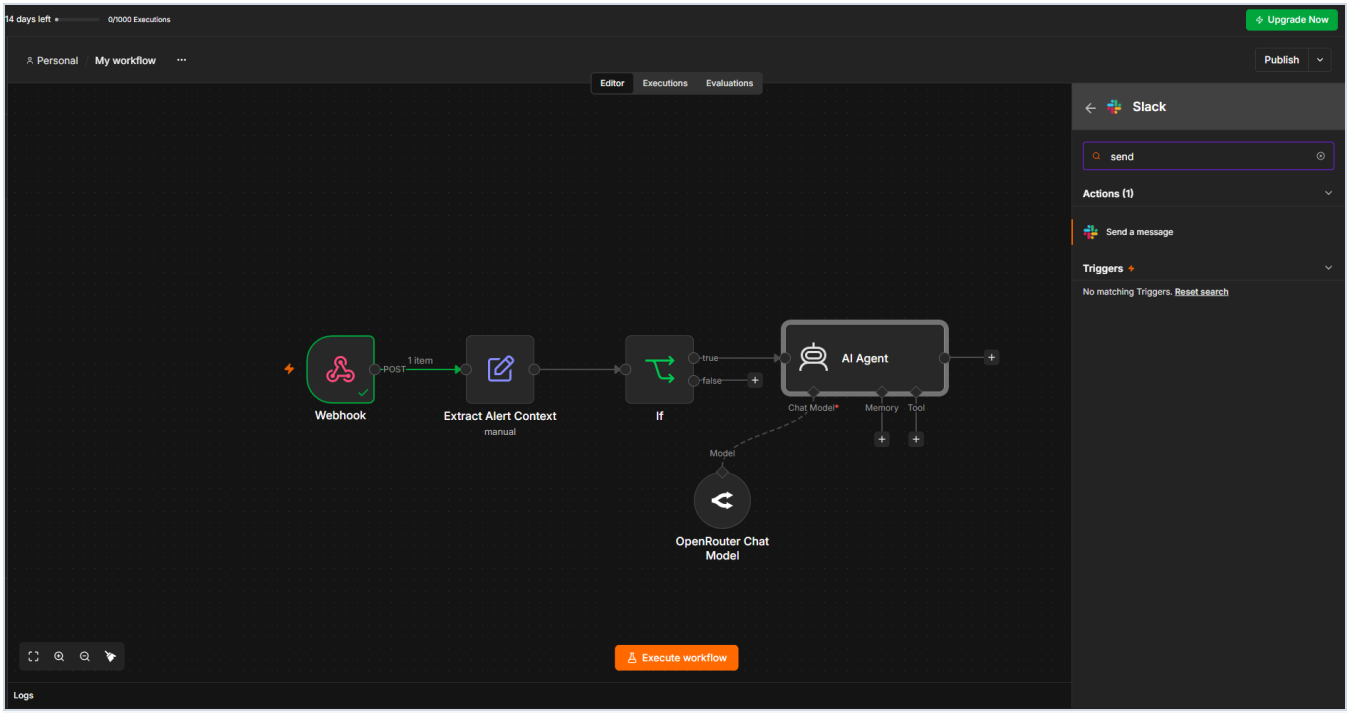


FIGURE 22

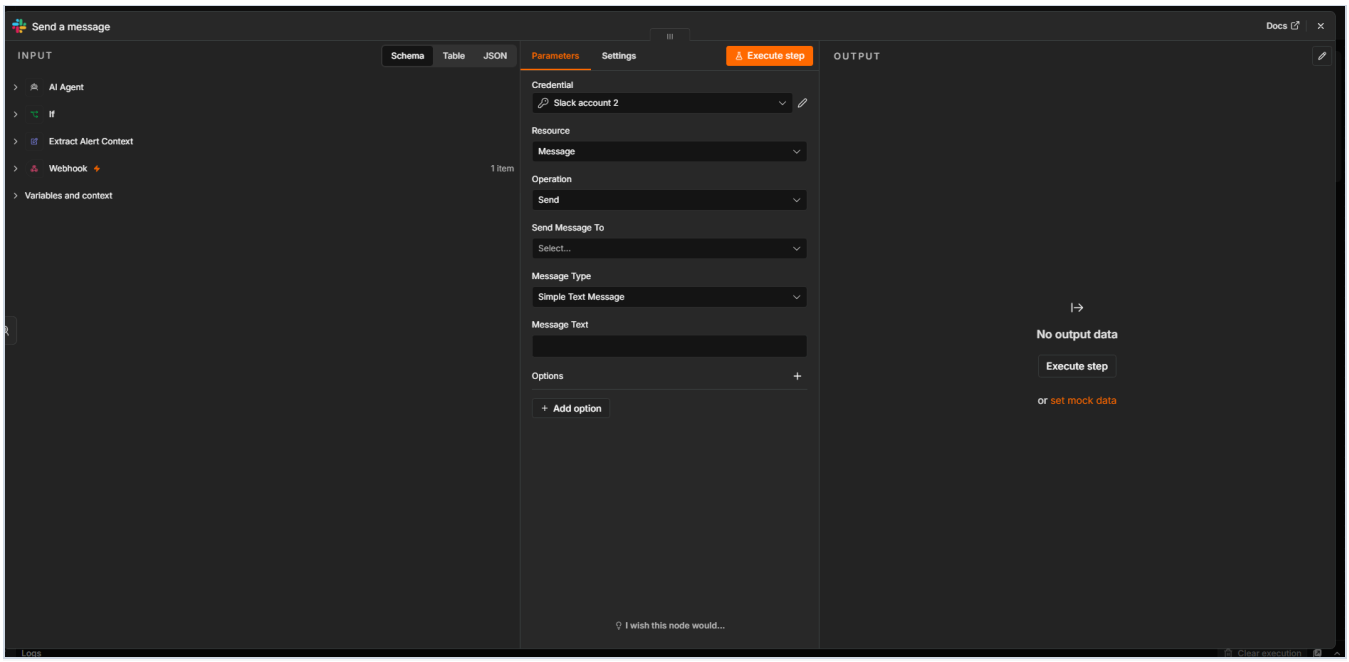


FIGURE 23

- To get the **Signing Secret**, I will return to the **Slack** platform, click **Basic Information**, and copy the **Signing Secret**

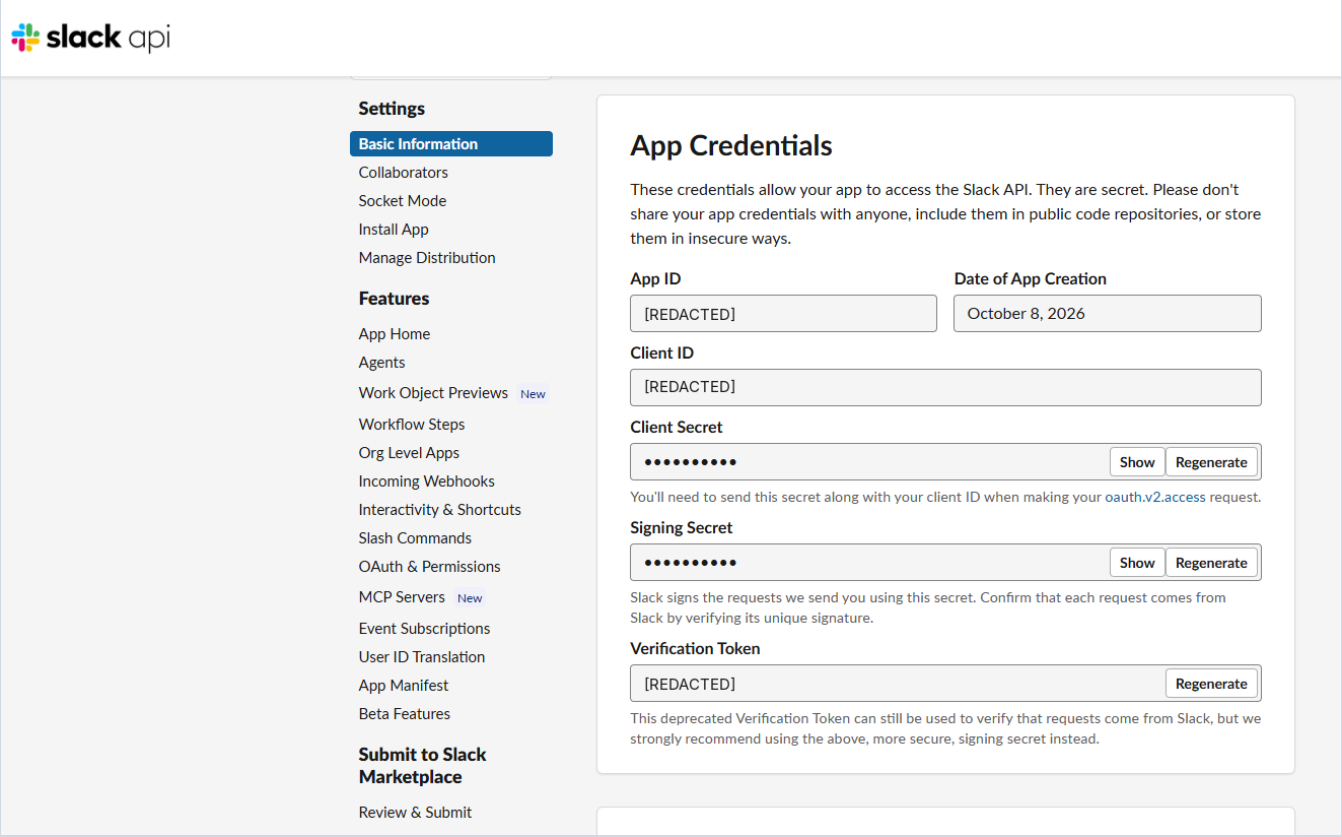


FIGURE 24

- Inside **Resource**, I will select **Message**, and inside **Operation**, I will select **Send**. For **Send Message To**, I will select **Channel**.
- Then, I will return to the **Slack workspace** and create a new channel called (**n8n-automation-security**-2).
- Inside **Channel**, I will select the channel I created, In the **Message Text** field, I will enter:

```
N8N EXPRESSION
{{$json.output}}
```

- This value means that the **AI Agent** returns its response in the `output` field. I will take this same output and send it to the Slack channel.
- There is also an important option: inside **Options**, I will select (**Include Link to Workflow**).

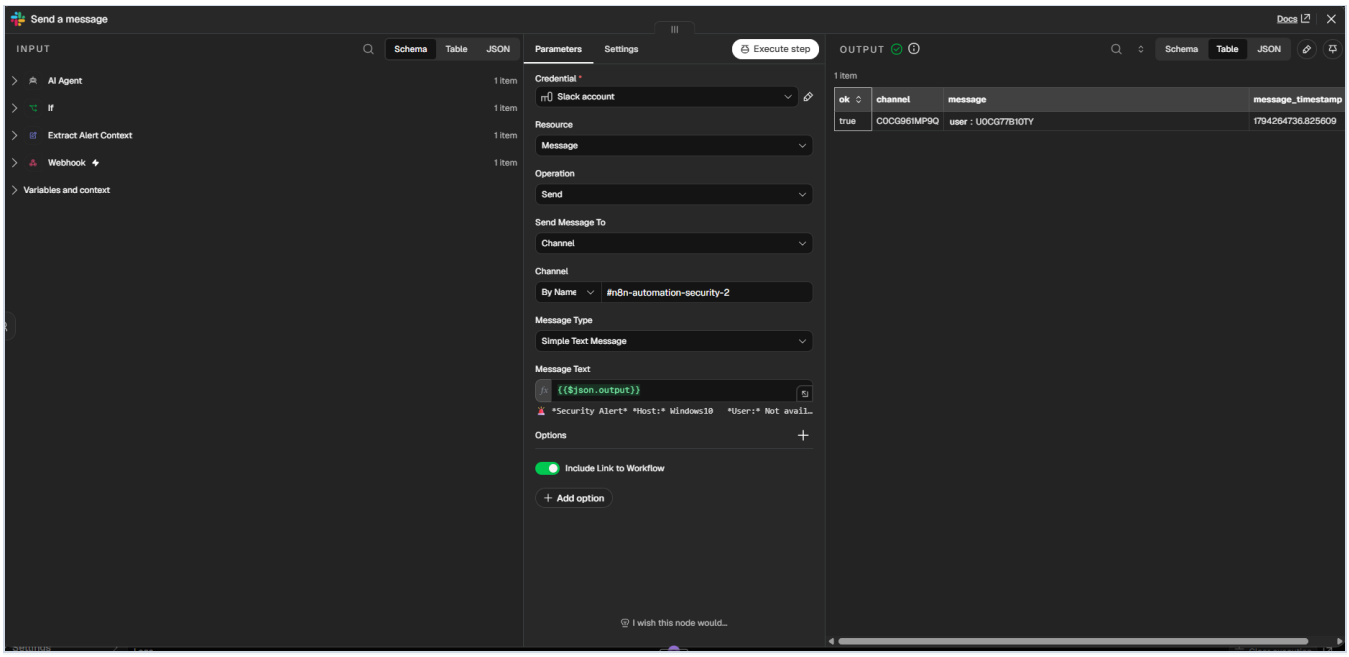


FIGURE 25

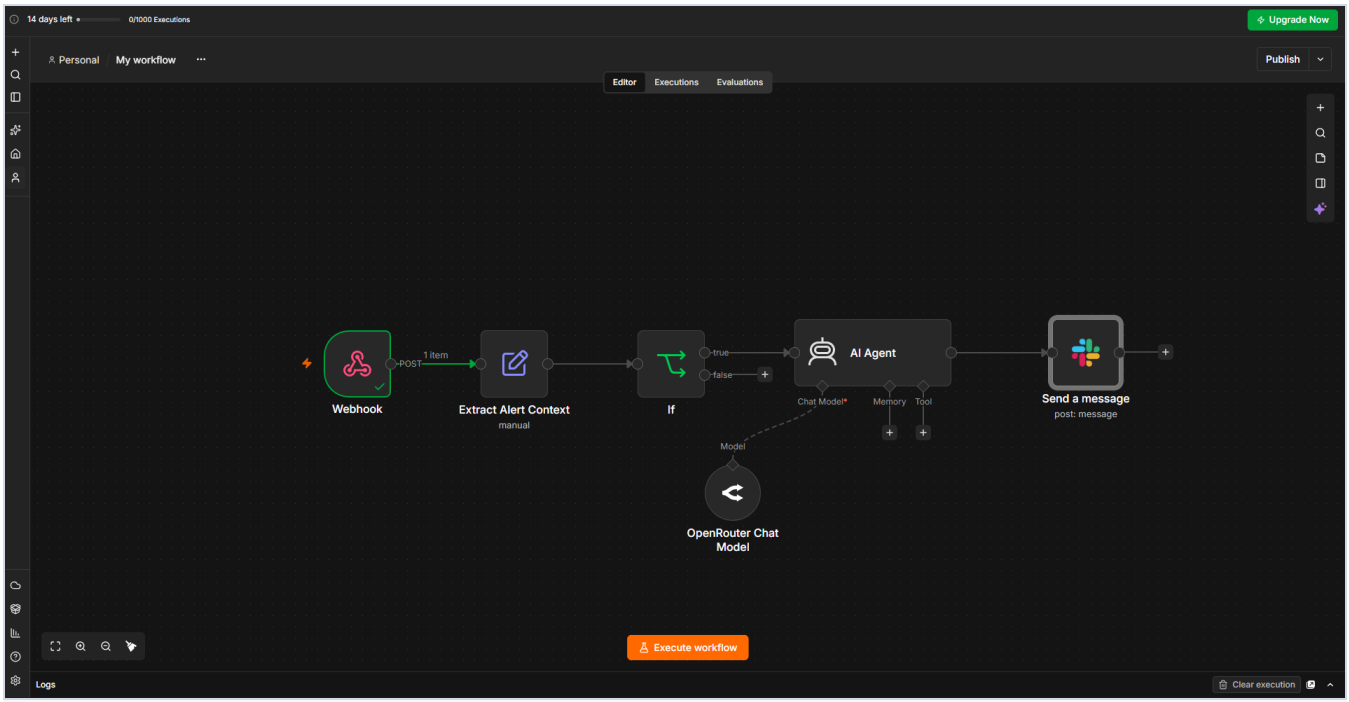


FIGURE 26

Explain the sentence "Include Link to Workflow"

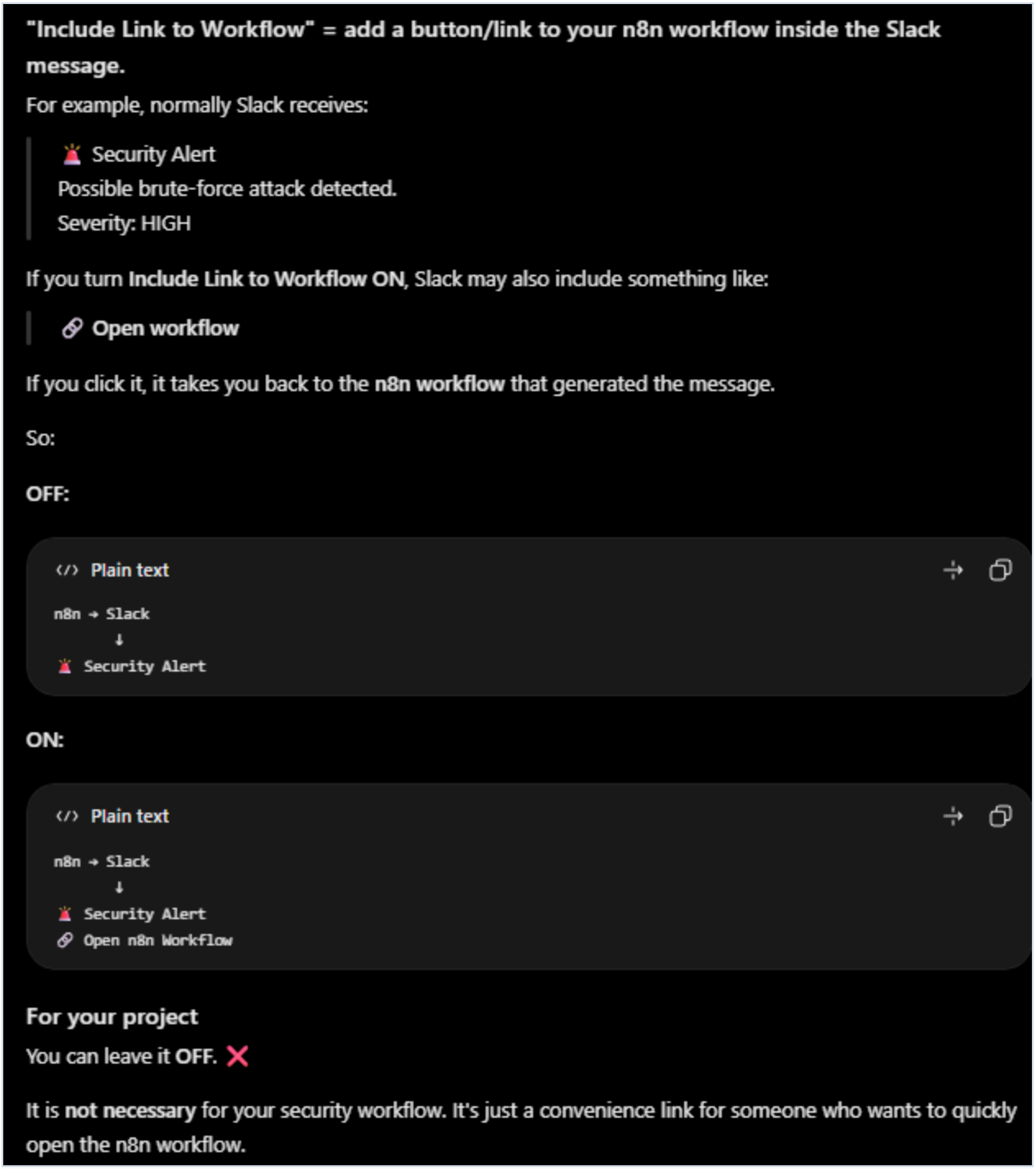


FIGURE 27

STEP 11 Custom n8n integration on Wazuh

- Now, we want to connect **Wazuh** with **n8n**. To do this, I will create a **custom configuration** called (`/var/ossec/integrations/custom-n8n`) that allows Wazuh to send its alerts directly to the **n8n webhook**.
- I will open **Wazuh** to get its IP address. Then, I will open **Kali Linux** and connect to Wazuh via SSH.
- After connecting to Wazuh through SSH, I will run the following command in Kali

```shell
sudo nano /var/ossec/integrations/custom-n8n
```

- Then, I will add the following code.

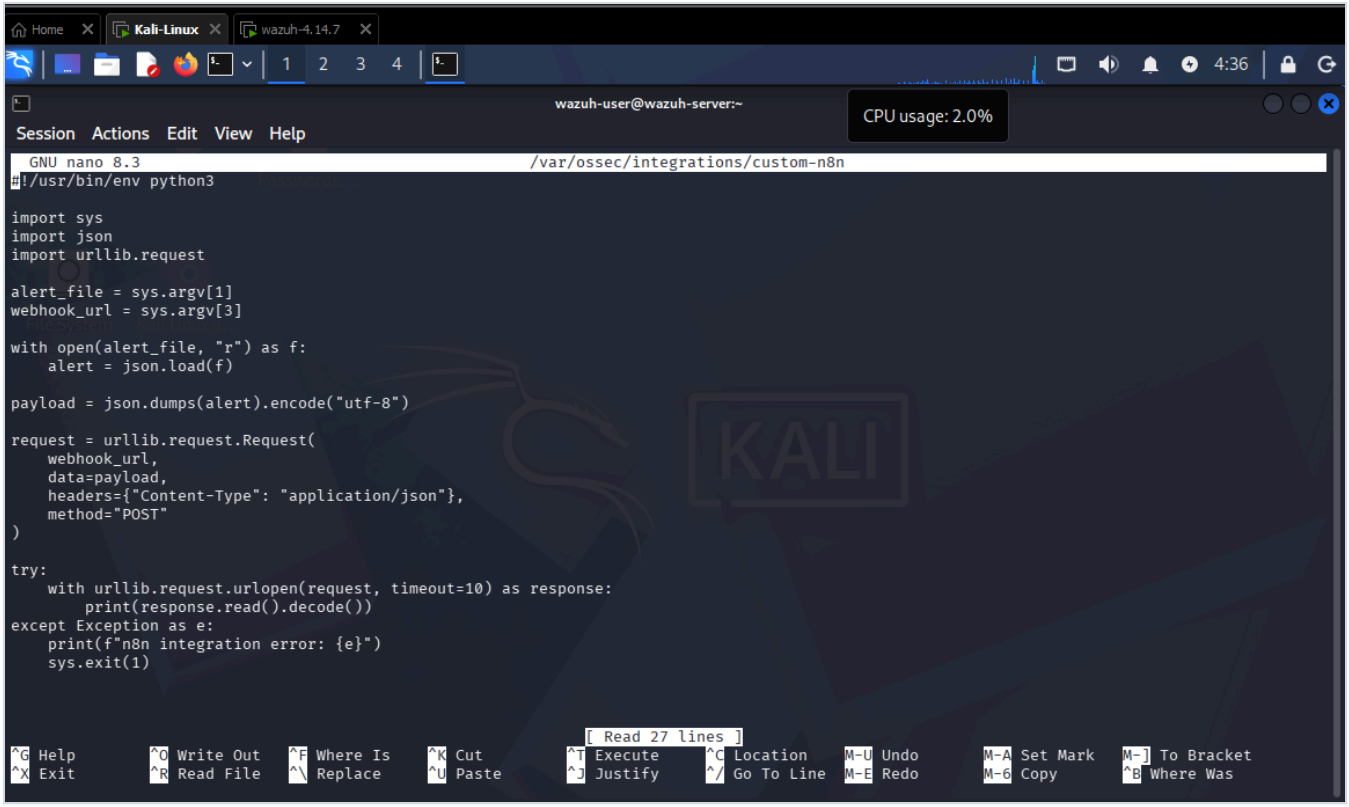


FIGURE 28

- This code does three things
  - Takes the alert file generated by Wazuh.
  - Reads it as a JSON file.
  - Sends the same alert as an HTTP POST request to the n8n webhook.

Wazuh Server

```
[wazuh-user@wazuh-server ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether REDACTED brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    altname ens32
    inet REDACTED/24 metric 1024 brd REDACTED scope global dynamic eth0
       valid_lft 1473sec preferred_lft 1473sec
    inet6 REDACTED/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
[wazuh-user@wazuh-server ~]$
```

FIGURE 29

- After creating the custom n8n integration script, I will change its permissions using the `chmod 750` command to make the script executable by the appropriate users.

```
[wazuh-user@wazuh-server ~]$ sudo chmod 750  /var/ossec/integrations/custom-n8n
[wazuh-user@wazuh-server ~]$
```

FIGURE 30

It is divided into three numbers:

Number	Applies to	Permissions
7	Owner	Read + Write + Execute
5	Group	Read + Execute
0	Others	No permissions

FIGURE 31

- I will change the owner of the file to `root` and the group to `wazuh` , so the custom integration has the appropriate ownership and is compatible with the Wazuh environment.

```
[wazuh-user@wazuh-server ~]$ sudo chown root:wazuh /var/ossec/integrations/custom-n8n
[wazuh-user@wazuh-server ~]$
```

FIGURE 32

- Now, I will open the **Wazuh configuration file** with write (`sudo nano /var/ossec/etc/ossec.conf`) to add the required code inside it & add Production URL : https://<your-n8n-instance>/webhook/<your-webhook-path>

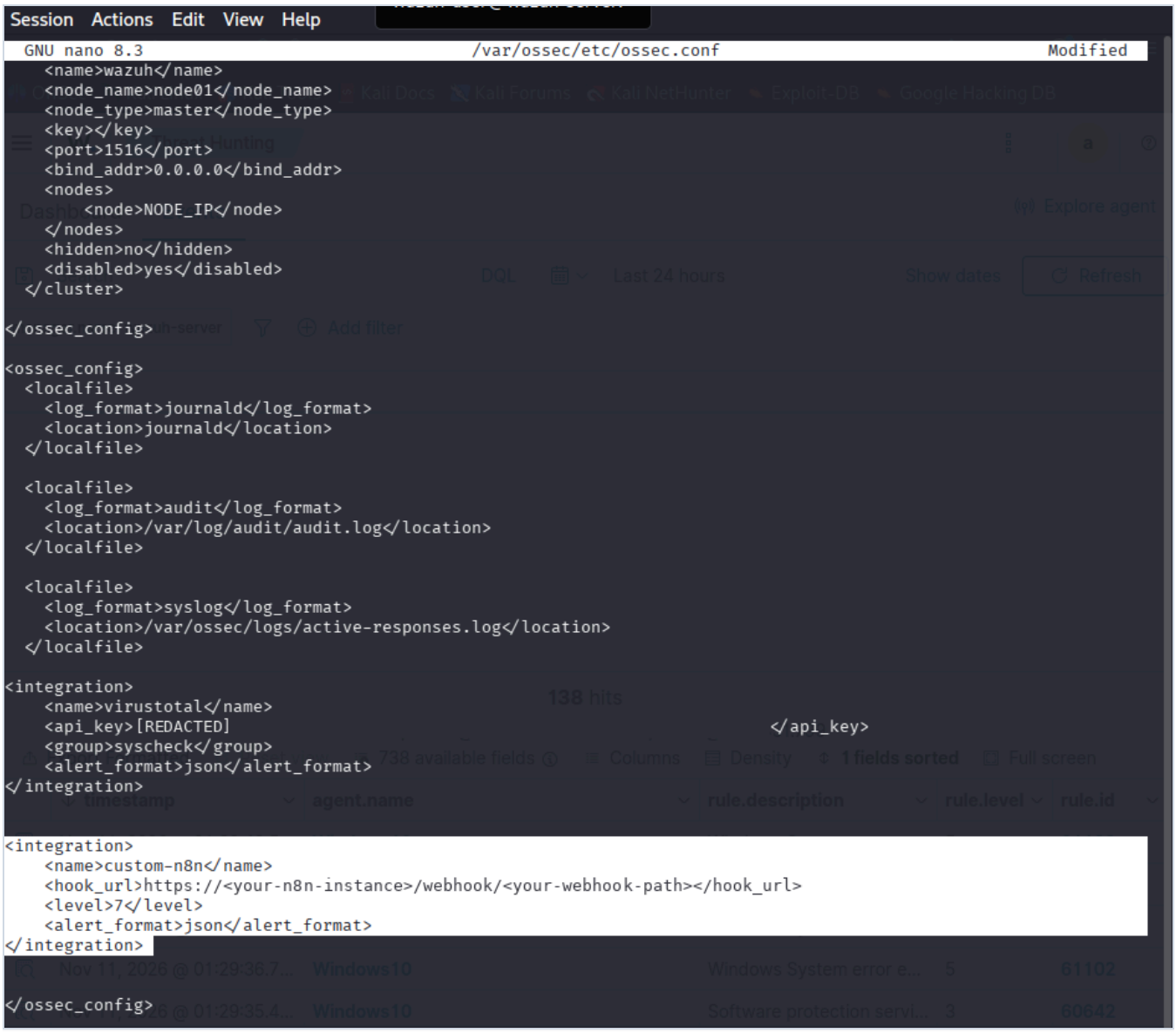

Session Actions Edit View Help
GNU nano 8.3 /var/ossec/etc/ossec.conf Modified
<name>wazuh</name>
<node_name>node01</node_name>
<node_type>master</node_type>
<key></key>
<port>1516</port>
<bind_addr>0.0.0.0</bind_addr>
<nodes>
<node>NODE_IP</node>
</nodes>
<hidden>no</hidden>
<disabled>yes</disabled>
</cluster>
</ossec_config>
<ossec_config>
<localfile>
<log_format>journald</log_format>
<location>journald</location>
</localfile>
<localfile>
<log_format>audit</log_format>
<location>/var/log/audit/audit.log</location>
</localfile>
<localfile>
<log_format>syslog</log_format>
<location>/var/ossec/logs/active-responses.log</location>
</localfile>
<integration>
<name>virustotal</name>
<api_key>[REDACTED]
</api_key>
<group>syscheck</group>
<alert_format>json</alert_format>
</integration>
<integration>
<name>custom-n8n</name>
<hook_url>https://<your-n8n-instance>/webhook/<your-webhook-path></hook_url>
<level>7</level>
<alert_format>json</alert_format>
</integration>
</ossec_config>

FIGURE 33

Production URL

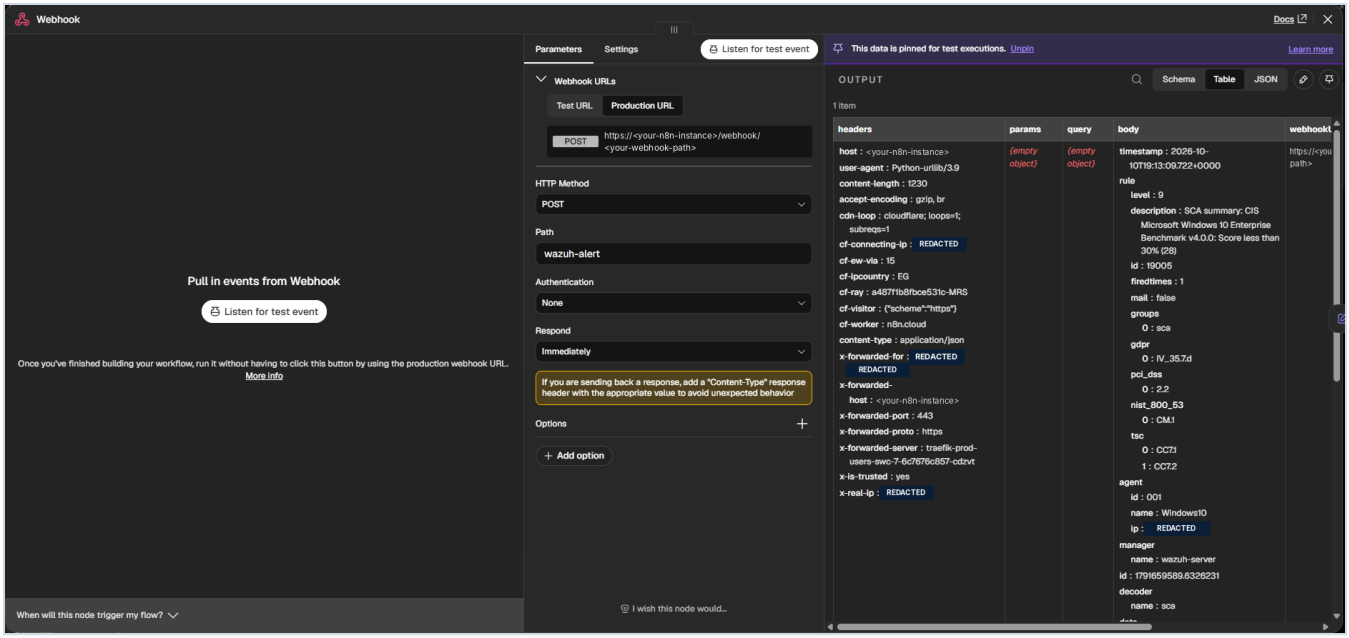


FIGURE 34

STEP 12 Restart the Wazuh manager

- then i restart wazuh manger

```
[wazuh-user@wazuh-server ~]$ sudo systemctl restart wazuh-manager
[wazuh-user@wazuh-server ~]$ sudo systemctl status wazuh-manager --no-pager
● wazuh-manager.service - Wazuh manager
     Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: disabled)
     Active: active (running) since Sat 2026-10-10 02:17:57 UTC; 50s ago
    Process: 7134 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
      Tasks: 212 (limit: 4647)
     Memory: 718.5M
        CPU: 52.506s
     CGroup: /system.slice/wazuh-manager.service
             ├─7197 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             ├─7198 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             ├─7199 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             ├─7202 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             ├─7205 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             ├─7225 /var/ossec/bin/wazuh-integratord
             ├─7245 /var/ossec/bin/wazuh-authd
             ├─7261 /var/ossec/bin/wazuh-db
             ├─7297 /var/ossec/bin/wazuh-execd
             ├─7313 /var/ossec/bin/wazuh-analysisd
             ├─7327 /var/ossec/bin/wazuh-syscheckd
             ├─7348 /var/ossec/bin/wazuh-remoted
             ├─7414 /var/ossec/bin/wazuh-logcollector
             ├─7435 /var/ossec/bin/wazuh-monitord
             ├─7455 /var/ossec/bin/wazuh-modulesd
             └─8553 /usr/bin/python3 /usr/bin/dnf list "installed|" grep sudo

Oct 10 02:17:49 wazuh-server env[7134]: Started wazuh-analysisd...
Oct 10 02:17:50 wazuh-server env[7134]: Started wazuh-syscheckd...
```

FIGURE 35

STEP 13 PowerShell test script and Slack channel

- Now, I will create a file called `Testscript.ps1` on the Windows machine. Then, I will edit the file and paste the following script:

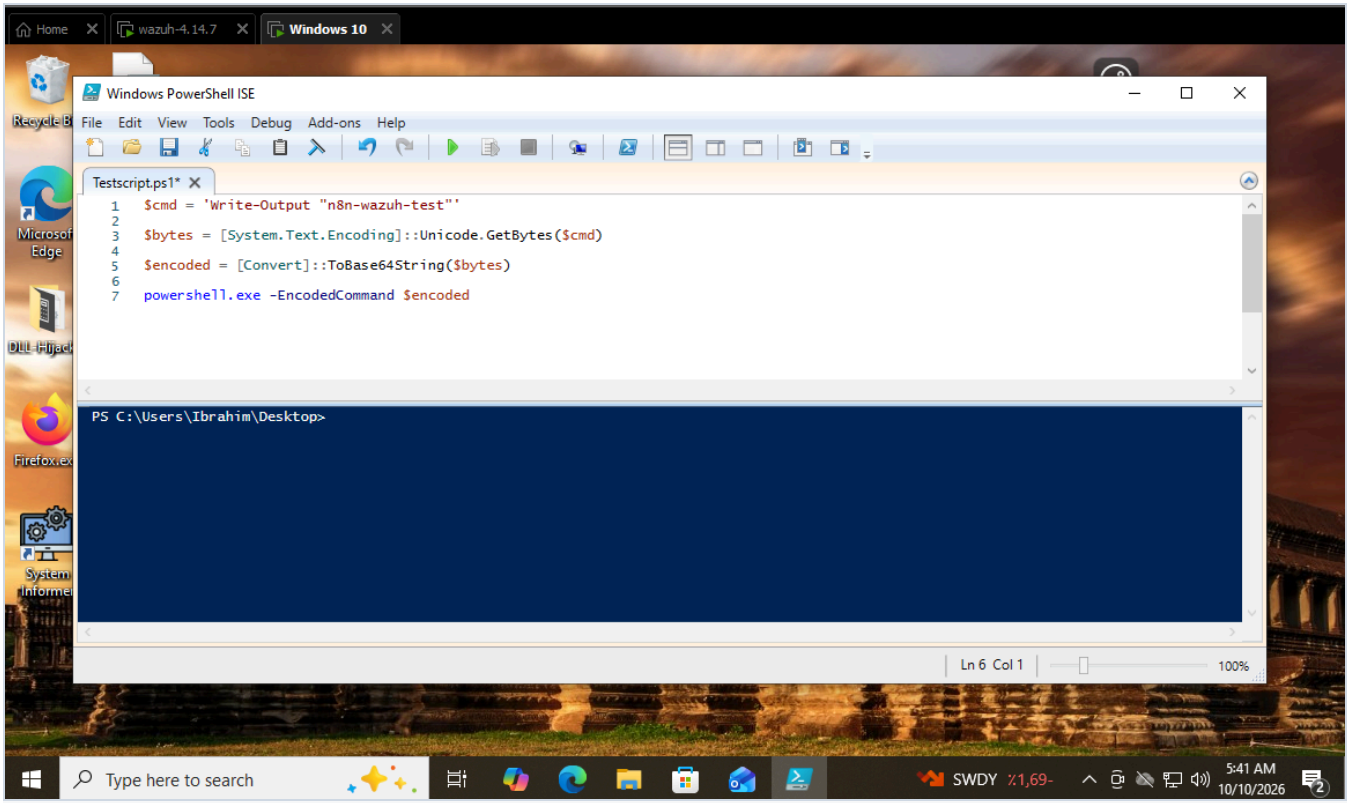


FIGURE 36

- The script works as follows:
  - **First line:** Stores the command `Write-Output "n8n-wazuh-test"` in the $cmd variable.
  - **Second line:** Converts the text stored in $cmd into bytes using Unicode encoding.
  - **Third line:** Takes those bytes and encodes them using **Base64**.
  - **Fourth line:** Executes the Base64-encoded command using PowerShell's `-EncodedCommand` option.
- I will use this test script to generate an encoded PowerShell execution event and verify how the **Wazuh → n8n → AI Agent** workflow analyzes it.
- Then, I will open the **Slack workspace** and go to the (**n8n-automation-security**-2) channel. I will type (`/invite`) in the chat and invite (**n8n-security-lab**), which is the bot I created.
- This allows the bot to access the channel so that whenever an alert is triggered, the AI-generated alert can be sent to **n8n-automation-security-2** through the Slack workflow.

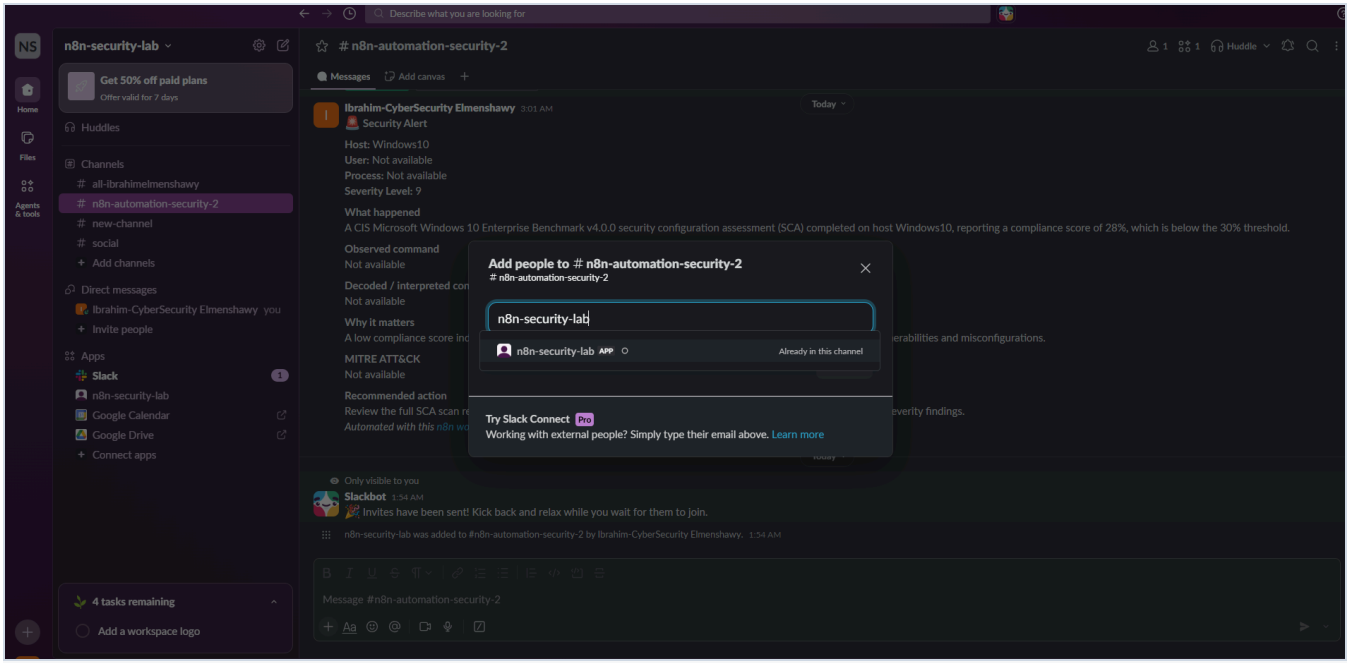


FIGURE 37

STEP 14 Publish the workflow

- I will click **Publish** to activate the workflow. Once the workflow is running, any alert that meets the configured conditions will be processed and sent to the **Slack channel**.

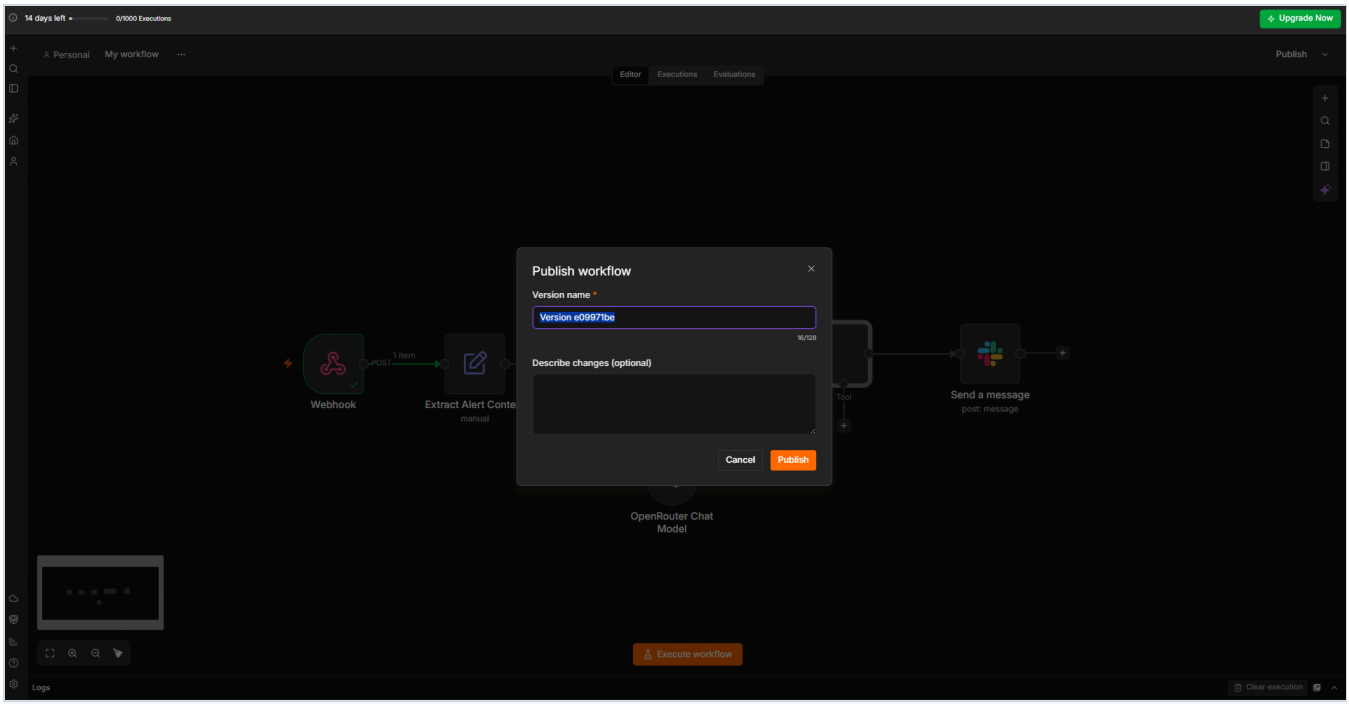


FIGURE 38

STEP 15 Run the test

- I returned back to the windows machine to run the script

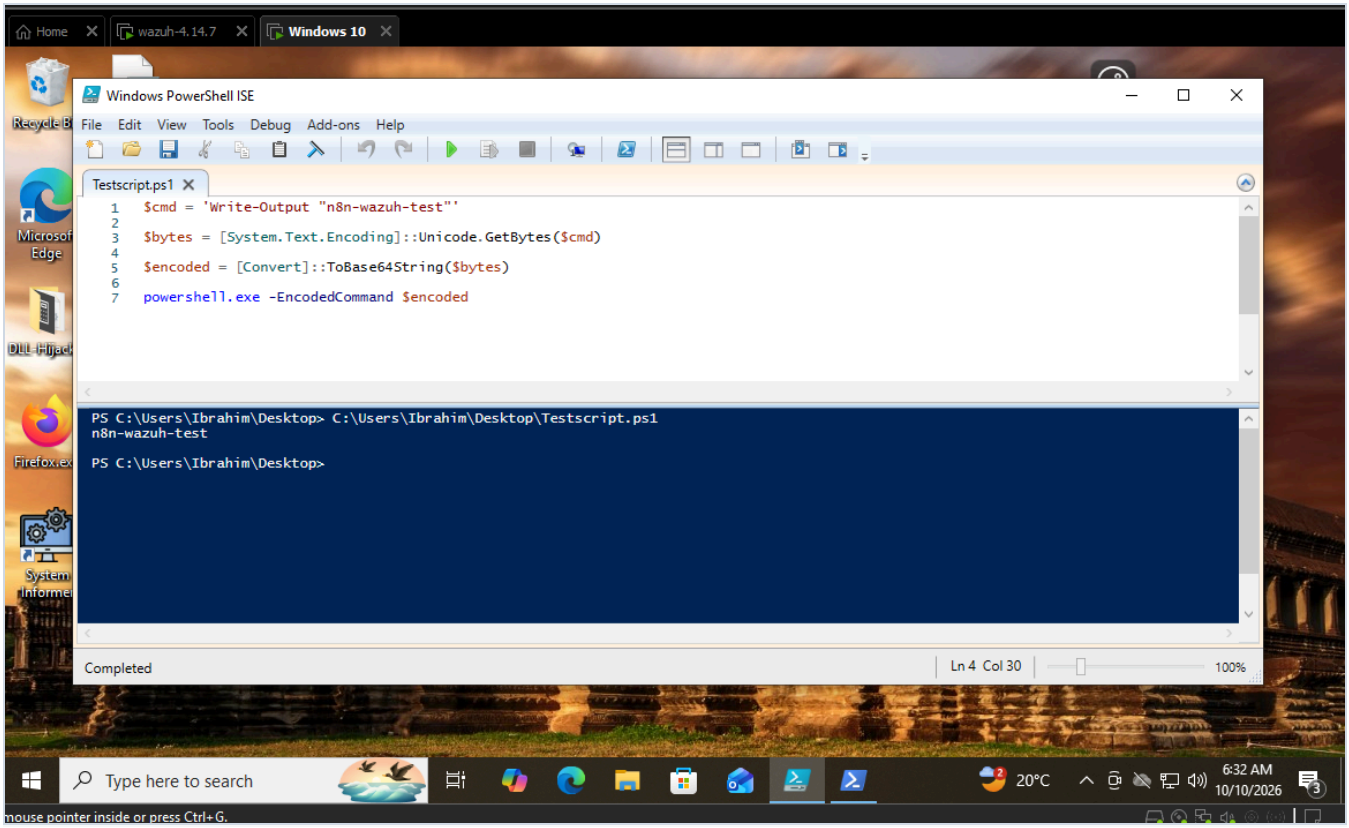


FIGURE 39

- When I execute the PowerShell test script, Wazuh detects the PowerShell activity, sends the security alert to n8n, and n8n automatically forwards a detailed notification to the Slack channel

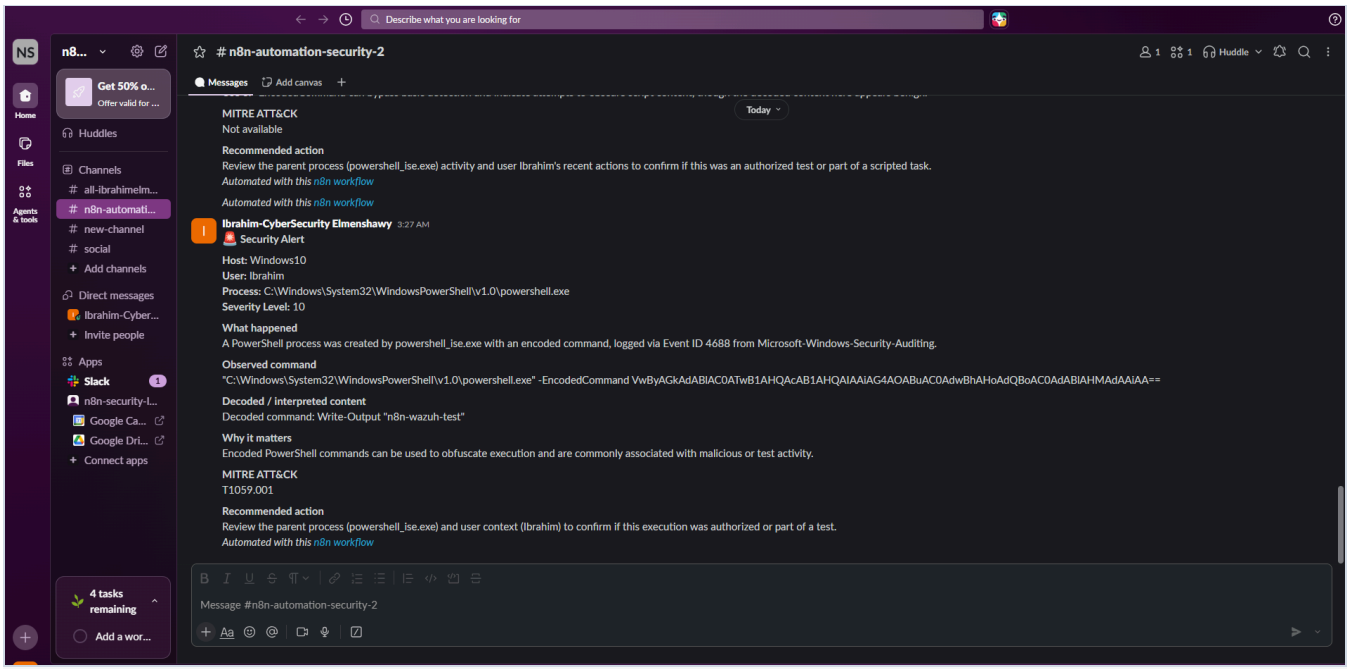

Describe what you are looking for
n8...
n8n-automation-security-2
Messages
Add canvas
Huddle
Get 50% o...
Offer valid for ...
Huddles
Channels
all-ibrahimelm...
n8n-automati...
new-channel
social
Add channels
Direct messages
Ibrahim-Cyber...
Invite people
Apps
Slack
n8n-security-l...
Google Ca...
Google Dri...
Connect apps
4 tasks remaining
Add a wor...
Today
MITRE ATT&CK
Not available
Recommended action
Review the parent process (powershell_ise.exe) activity and user Ibrahim's recent actions to confirm if this was an authorized test or part of a scripted task.
Automated with this n8n workflow
Automated with this n8n workflow
Ibrahim-CyberSecurity Elmenshawy 3:27 AM
Security Alert
Host: Windows10
User: Ibrahim
Process: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
Severity Level: 10
What happened
A PowerShell process was created by powershell_ise.exe with an encoded command, logged via Event ID 4688 from Microsoft-Windows-Security-Auditing.
Observed command
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -EncodedCommand VwByAGkAdABlAC0ATwB1AHQAcAB1AHQAIAAiAG4AOABuAC0AdwBhAHoAdQBoAC0AdABlAHMAdAAiAA==
Decoded / interpreted content
Decoded command: Write-Output "n8n-wazuh-test"
Why it matters
Encoded PowerShell commands can be used to obfuscate execution and are commonly associated with malicious or test activity.
MITRE ATT&CK
T1059.001
Recommended action
Review the parent process (powershell_ise.exe) and user context (Ibrahim) to confirm if this execution was authorized or part of a test.
Automated with this n8n workflow
Message #n8n-automation-security-2

FIGURE 40

SECTION 02

Problems

PROBLEM 1 PowerShell execution policy

- i Solved it with run

```powershell
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

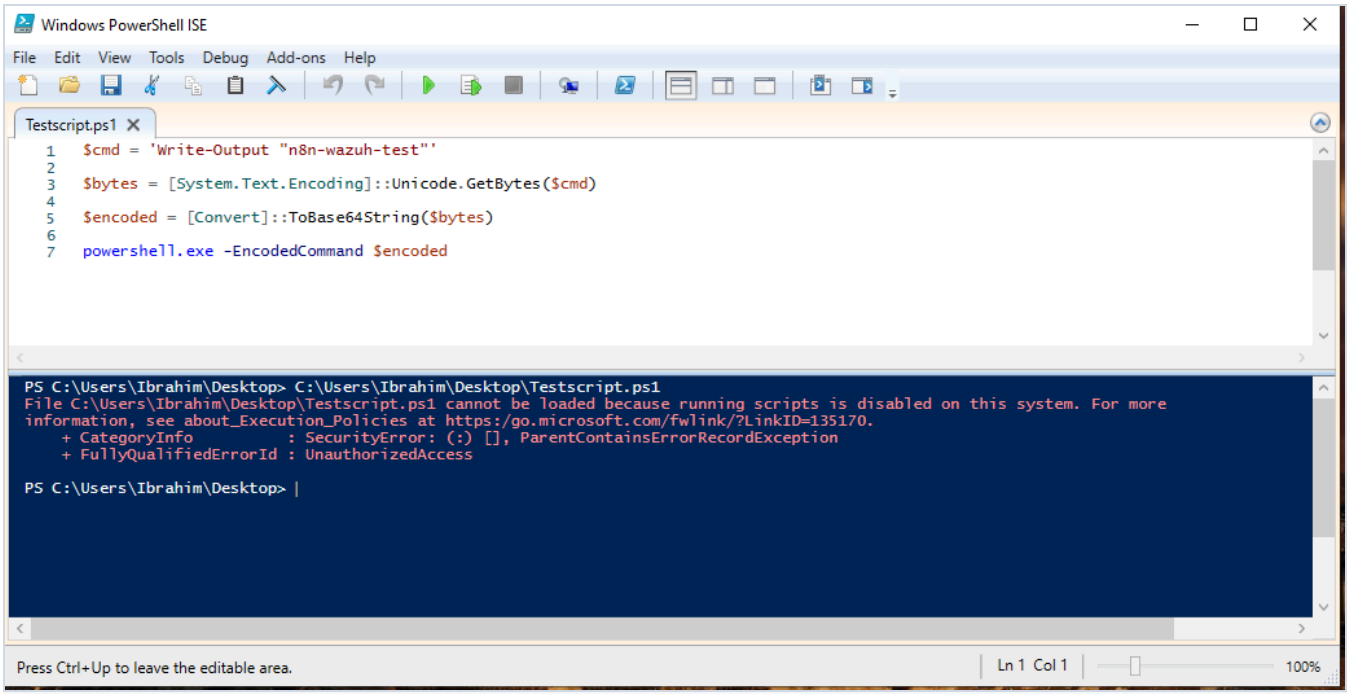


FIGURE 41

PROBLEM 2

Alert level thresholds

1. Wazuh decides the alert level

When an event happens, Wazuh analyzes it using its rules.

For example:

```
Plain text
Windows event
     ↓
Wazuh rule 19005
     ↓
Alert level = 9
```

The **9 belongs to the alert.** It is not something you created in n8n.

---

2. Wazuh's integration setting decides what gets sent to n8n

You configured:

```
XML
<level>7</level>
```

This means:

> **"Send every Wazuh alert whose level is 7 or higher to n8n."**

So:

```
Plain text
Level 6  → ❌ Not sent
Level 7  → ✅ Sent
Level 8  → ✅ Sent
Level 9  → ✅ Sent
Level 10 → ✅ Sent
...
Level 15 → ✅ Sent
```

Therefore, your level-9 alert **correctly reached n8n.**

FIGURE 42

3. n8n's IF node makes another decision

Your IF node was configured as:

```
Plain text
rule.level >= 10
```

But the alert was:

```
Plain text
rule.level = 9
```

So n8n calculated:

```
Plain text
9 >= 10
```

Result:

```
Plain text
FALSE ❌
```

Nothing was broken. The workflow was doing exactly what you told it to do.

FIGURE 43

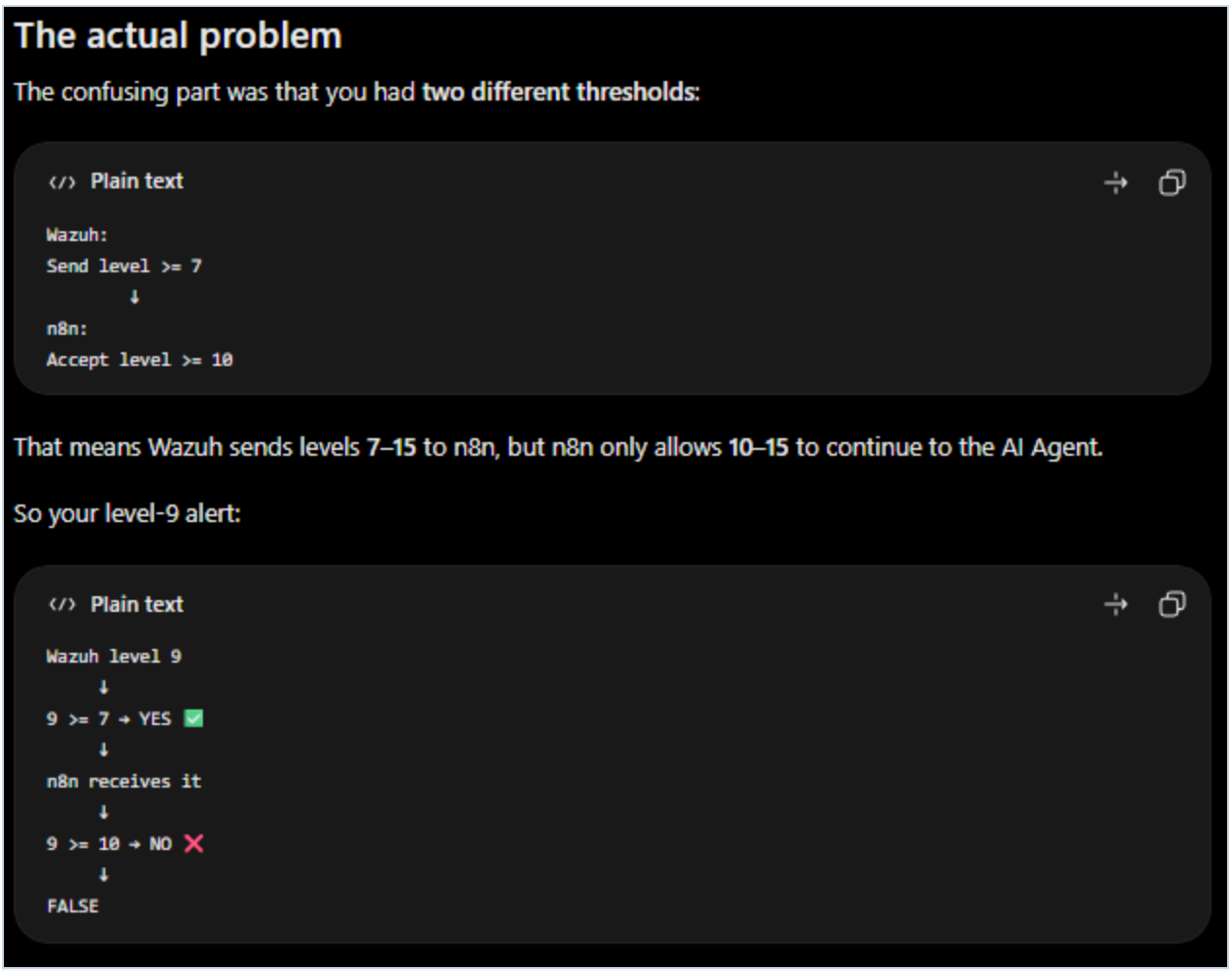

The actual problem
The confusing part was that you had two different thresholds:
Plain text
Wazuh:
Send level >= 7
n8n:
Accept level >= 10
That means Wazuh sends levels 7–15 to n8n, but n8n only allows 10–15 to continue to the AI Agent.
So your level-9 alert:
Plain text
Wazuh level 9
9 >= 7 → YES
n8n receives it
9 >= 10 → NO
FALSE

FIGURE 44

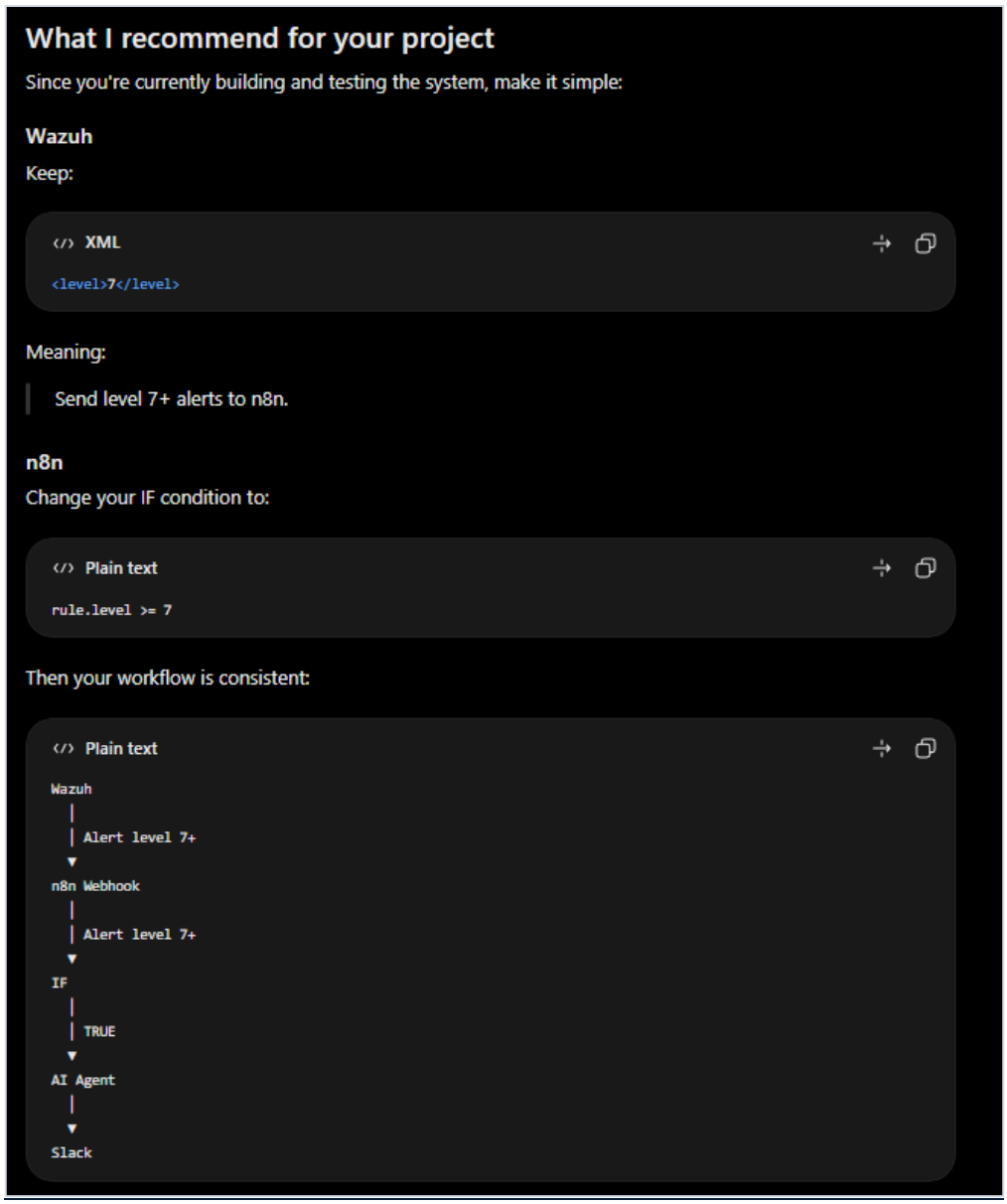


FIGURE 45

NOTE · FINAL CONFIGURATION

Figures 45 and 46 show the troubleshooting suggestion received at the time, which proposed changing the IF condition to `rule.level >= 7`. That suggestion was **not** adopted. The final implemented configuration keeps two separate thresholds:

- **Wazuh integration threshold:** `<level>7</level>` – Wazuh forwards alerts of level 7 or higher to n8n.
- **Final n8n IF threshold:** `{{ $json.rule_level }} >= 9` – n8n forwards only alerts of level 9 or higher to the AI Agent.

Alerts of level 7 and 8 therefore reach n8n but follow the False Branch and are not sent to the AI Agent, unlike the "7 → AI" lines shown in Figure 46.

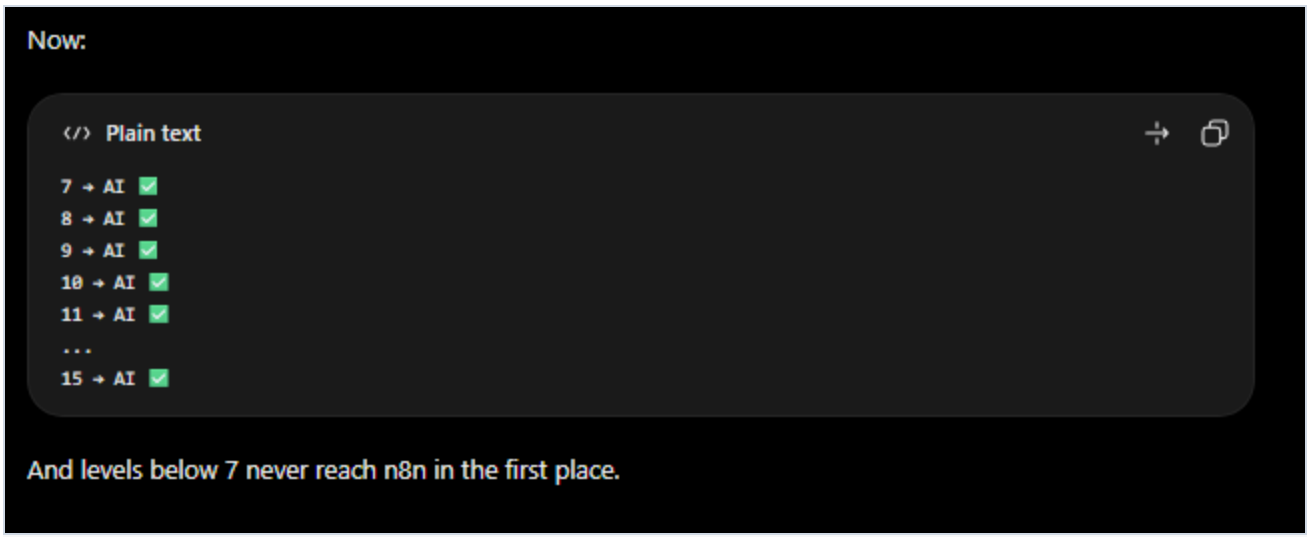


FIGURE 46

PROBLEM 3 Process creation and command-line auditing

What was the problem?

- We wanted Wazuh to detect PowerShell execution on the Windows machine and automatically send a detailed security alert to Slack through n8n.
- However, Windows was not recording the process creation and command-line details needed by Wazuh.

Solution

- 1- Windows 10 – PowerShell commands
- 2- Windows – PowerShell test
- 3- Wazuh – Create the custom detection rule
- 4- Wazuh – Validate and restart
- 5- Wazuh – Check the generated alerts
- 6- n8n – Webhook and workflow

1- Windows 10 – PowerShell commands

- We opened PowerShell as Administrator and ran these commands

Command 1 – Enable Process Creation Auditing

- Command ---->

```powershell
auditpol /set /subcategory:"Process Creation" /success:enable
```

- This enables Windows to record when a new process is created.

Command 2 – Enable Command-Line Auditing

- Command ---->

```powershell
reg add "HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit" /v Process CreationIncludeCmdLine_Enabled /t REG_DWORD /d 1 /f
```

- This enables Windows to include the command line in process creation events.

Command 3 – Verify Process Creation Auditing

- Command ---->

```powershell
auditpol /get /subcategory:"Process Creation"
```

- We checked that Success auditing was enabled.

2- Windows – PowerShell test

- I ran the script included with Windows , It launched PowerShell with an encoded command that decoded to : `Write-Output "n8n-wazuh-test"`
- The resulting process creation event was Windows Security Event ID 4688.
- The command line contained `-EncodedCommand` , which allowed us to create a custom Wazuh detection rule.

3- Wazuh – Create the custom detection rule

- On the Wazuh server, we edited :

```shell
sudo nano /var/ossec/etc/rules/local_rules.xml
```



FIGURE 47

- We added this rule :



FIGURE 48

4- Wazuh – Validate and restart

- After saving the XML file, we ran
  - Validate the configuration : `sudo /var/ossec/bin/wazuh-analysisd -t`
- Restart the Wazuh manager : `sudo systemctl restart wazuh-manager`
- Check the service status : `sudo systemctl status wazuh-manager`

- We checked that the service was `active (running)` .

5- Wazuh – Check the generated alerts

- We used : `sudo tail -f /var/ossec/logs/alerts/alerts.json`
  - This allowed us to monitor incoming alerts.
- Then we checked specifically for our custom rule

```shell
sudo grep '"id":"100500"' /var/ossec/logs/alerts/alerts.json | tail -3
```

- We successfully found the alert with:
  - Rule ID: `100500`
  - Severity: `12`
  - Description: `PowerShell EncodedCommand executed on Windows endpoint`

> **NOTE · TWO VERSIONS OF RULE 100500**
>
> The documentation contains two different captured versions of custom rule `100500`. They are kept as recorded and are not merged:
>
> - **Figure 47** (local_rules.xml on the Wazuh server): `level="10"` with MITRE technique `T1059.001`. The Slack alert in Figure 40 matches this version (Severity Level 10, MITRE ATT&CK T1059.001).
> - **Figure 48:** `level="12"` without the `<mitre>` block. The alerts.json check above matches this version (Severity 12).
>
> These are two separate rule configurations captured during the project, not two rules active at the same time. Which version is currently deployed is not recorded. Both values pass the final n8n IF condition `rule_level >= 9` (10 ≥ 9 and 12 ≥ 9), so either version reaches the AI Agent.

6- n8n – Webhook and workflow

- We used this production webhook :
  - https://<your-n8n-instance>/webhook/<your-webhook-path>
- We verified that

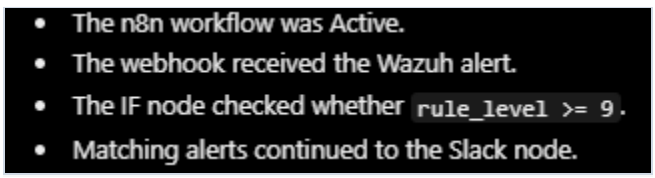


FIGURE 49

SECTION 03

Important Note

What is a webhook?

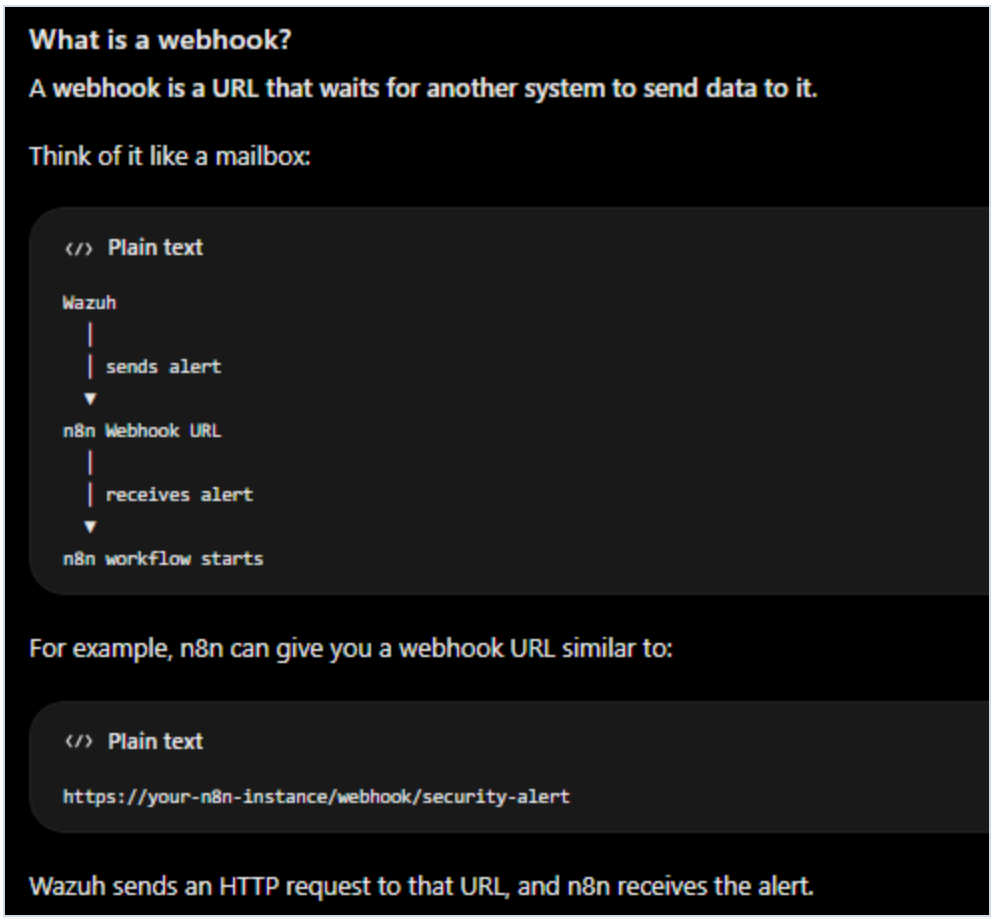


FIGURE 50

Why POST?

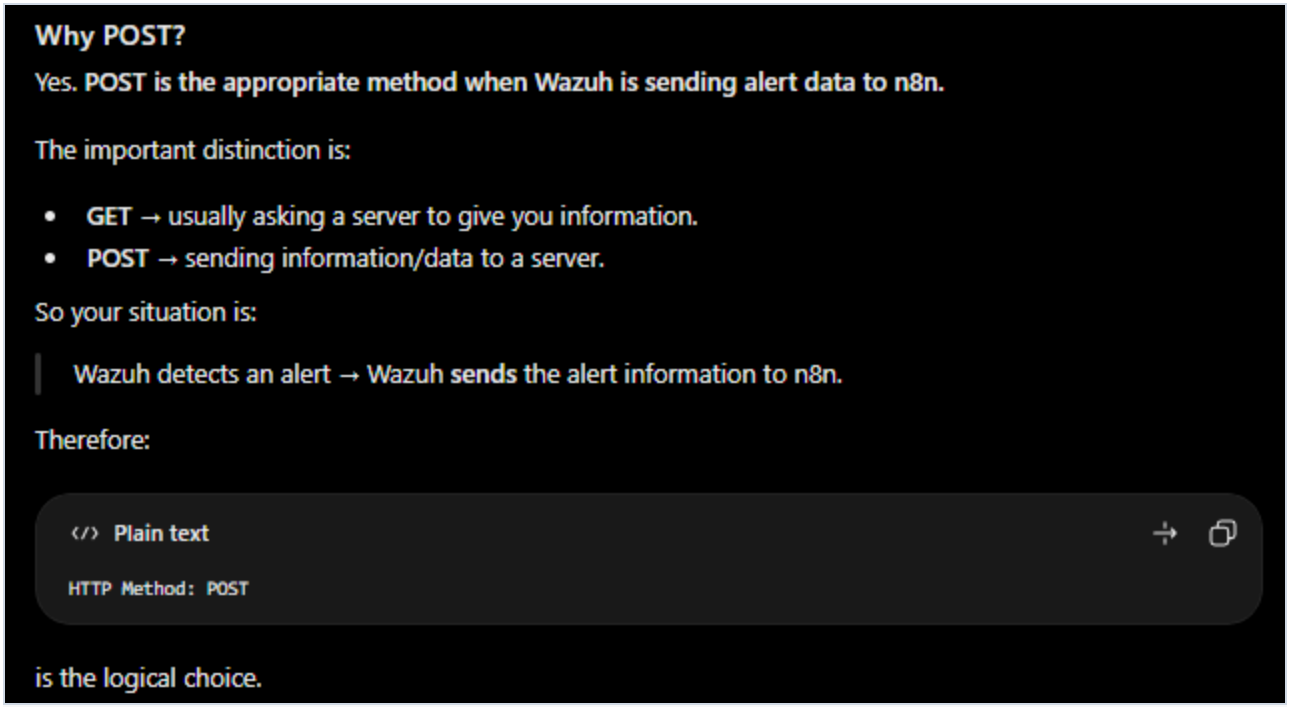


FIGURE 51

Overall

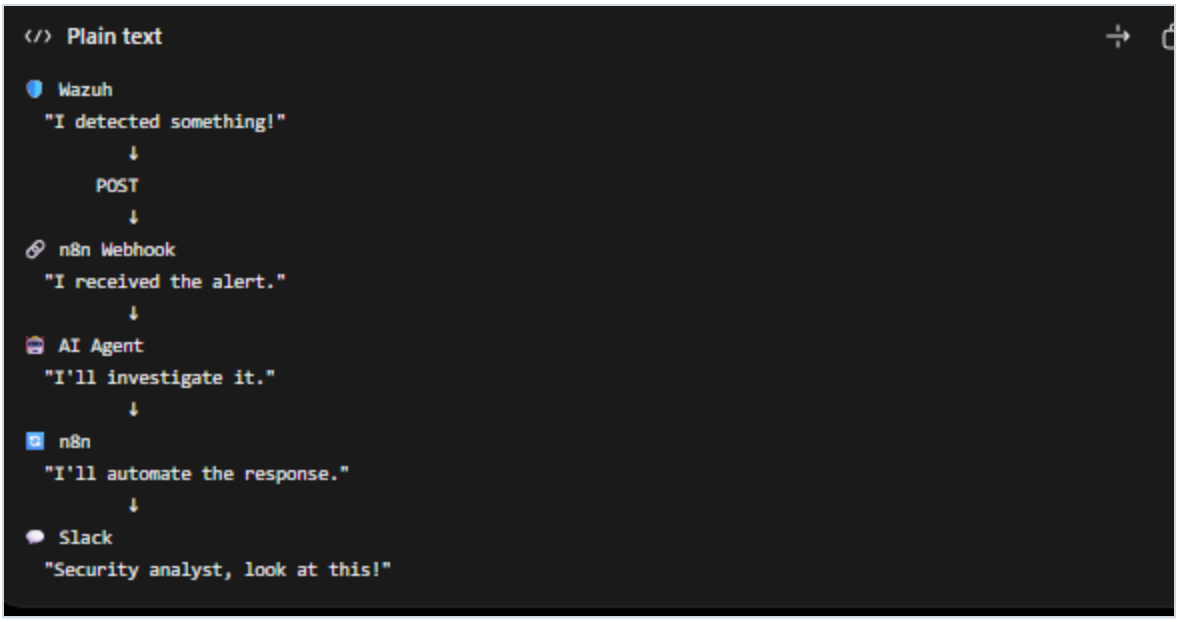


FIGURE 52

Explain URLS

- Build and test the workflow using the Test URL → once everything works, activate the workflow → configure Wazuh to send alerts to the Production URL.

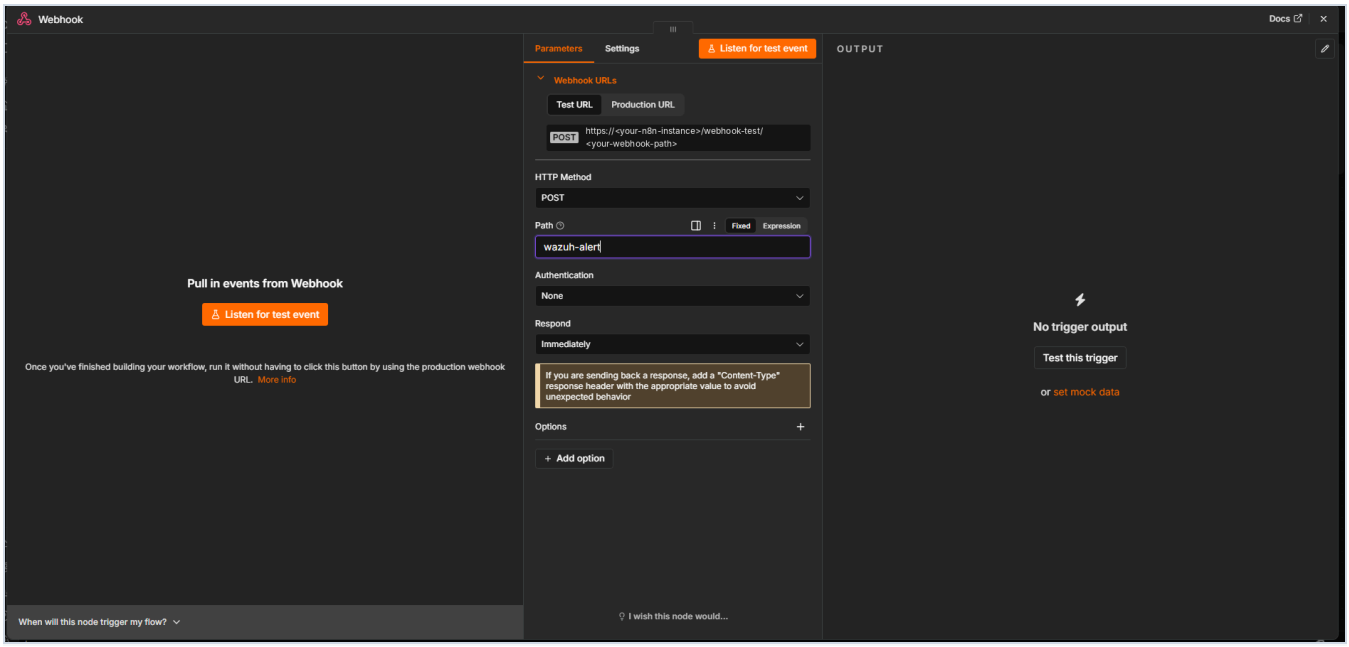


FIGURE 53

Test URL — while you're building

Use the **Test URL** while developing and testing your workflow.

In your screenshot, it looks like:

Plain text

```
POST
https://.../webhook-test/wazuh-alert
```

When you click **"Listen for test event"**, n8n starts listening for a request on that URL.

Then you can send a test alert to it and see the data appear in n8n.

So your development flow is:

Plain text

```
Wazuh / Test request
        ↓
   Test URL
        ↓
      n8n
        ↓
   AI Agent
        ↓
     Slack
```

FIGURE 54

Production URL — when your workflow is finished

The **Production URL** is what you'll configure Wazuh to use when your workflow is actually active.

It will look something like:

Plain text

```
POST
https://.../webhook/wazuh-alert
```

Your final setup becomes:

Plain text

```
Wazuh
  |
  | POST
  ▼
Production Webhook URL
  |
  ▼
n8n
  |
  ▼
AI Agent
  |
  ▼
Slack
```

FIGURE 55